



From benchmarks to deployment: a comprehensive review of agentic AI evaluation

Tanzila Kehkashan^{1,2} · Muhammad Abdullah² · Ahmad Sami Al-Shamayleh³ · Nikola Ivković⁴ · Nor Azman Ismail¹ · Sharifah Sakinah Syed Ahmad⁵ · Abdul Rehman² · Adnan Akhunzada⁶ 

Received: 12 January 2026 / Accepted: 8 April 2026 / Published online: 24 April 2026
© The Author(s) 2026

Abstract

This review systematically examines evaluation methodologies for agentic AI systems, agentic AI systems capable of multi-step planning, tool usage, and environmental interaction across diverse domains. Current evaluation practices exhibit a critical disconnect between benchmark performance and deployment viability, where agents achieving high scores on standardized benchmarks frequently fail in real world applications due to fundamental inadequacies in assessment methodologies that prioritize task completion over deployment critical dimensions such as cost efficiency, safety compliance, maintainability, and workflow integration. We critically analyze 15 major agent benchmarks including AgentBench, WebArena, SWE-bench, PaperBench, MLGym, BrowserGym, HumanEval, MBPP, GAIA, ToolBench, Terminal-Bench, Mind2Web, ALFWorld, BabyAI, and HotPotQA, examining their methodologies, metrics, datasets, and application domains with software development serving as our primary case study. Our analysis reveals that evaluation methodology not model capability constitutes the primary bottleneck limiting reliable agent deployment. We demonstrate how test passing metrics systematically ignore code quality, security vulnerabilities, and integration complexity while binary success metrics obscure planning coherence, resource efficiency, and safety violations (0/15 benchmarks integrate security or safety into scoring. This review provides a cross-domain taxonomy exposing evaluation inadequacies, trajectory level evaluation frameworks addressing cost reproducibility validity trade-offs, systematic identification of metric insufficiencies including absence of safety-aware and cost aware scoring, and synthesis of emerging evaluation paradigms with adoption barriers. Quantitatively, 0/15 benchmarks integrate safety or security into scoring, 0/15 include cost-efficiency metrics in their primary evaluation protocol, and 13/15 rely exclusively on binary success measures confirming that evaluation methodology, not model capability, is the primary bottleneck to reliable deployment. Progress toward trustworthy agentic AI fundamentally depends on evolving evaluation infrastructure beyond binary metrics toward comprehensive, multidimensional assessment frameworks that capture these deployment-essential dimensions.

Tanzila Kehkashan, Muhammad Abdullah, Abdul Rehman and Adnan Akhunzada contributed equally to this work.

Extended author information available on the last page of the article

Keywords Agent benchmarking · Evaluation frameworks · Multi-step reasoning · Tool-use assessment · Trajectory-based evaluation · Code generation evaluation

1 Introduction

Artificial intelligence has been radically changed in the last decade to abandon task-specific models and embrace agentic systems, often with autonomous decision-making and multi-step planning and interaction with complex environments. The shift has occurred mainly due to the development of large language models (LLMs) which have proven to be capable of unprecedented natural language understanding, logical thinking and code creation in a way that was previously unattainable Wang et al. (2024a). Present-day agentic AI systems operate today in a wide range of high-stakes tasks such as autonomous software engineering, automation of web-based tasks, scientific discovery, and enterprise decision support, and their capability to break down high-level goals, use external tools, and adapt to dynamic situations constitutes a shift in paradigm to the traditional AI system architecture (Wang et al. 2024b; Hou et al. 2024). Unlike classical AI models assigned to isolated tasks with fixed inputs and outputs, agentic AI systems act in a goal-directed manner, maintain memory, perceive their surroundings, and improve continuously through feedback. This review primarily focuses on LLM-based agentic systems agents that leverage large language models for planning, reasoning, and tool invocation as these represent the dominant paradigm in current benchmarks (AgentBench, WebArena, SWE-bench). While we acknowledge RL-based autonomous agents and hybrid neuro-symbolic systems, our evaluation analysis specifically targets LLM-based agents due to their prevalence in the 15 examined benchmarks and their immediate deployment relevance in software development, web automation, and research assistance domains.

The systems combine the planning algorithms, tool-use capabilities and multi-step reasoning to achieve the goals that might involve hundreds or thousands of sequential steps over longer periods of time horizons (Wang et al. 2024a). The application of LLM-based agents in practice revealed that there is a fundamental need to prioritize the reliability, safety, and intentionality of human goals, especially in the case of the autonomy of these systems in areas where mistakes may have serious technical, financial, or ethical implications on human lives and welfare at large (Terragni et al. 2025). This functional complexity implies the need to evaluate at a level beyond mere accuracy measures, to include extensive measures of the consistency of planning, ability to recover errors, resource use efficiency and behavioral robustness in the face of uncertainty.

The adoption and spread of agentic AI systems has given rise to an ever growing ecosystem of benchmarks and evaluation regimes that seek to quantify the performance of agents in a variety of dimensions. Over fifty different agent benchmarks have been proposed by the research community, since 2021, and they include web navigation, software development, scientific reasoning, embodied robotics, general-purpose task completion, etc Hou et al. (2024). Nevertheless, this has been growing in a highly disjointed fashion, which has led to high levels of fragmentation, when it comes to evaluation approaches, definitions of metrics, environmental presuppositions, and ability taxonomies. Current benchmarks are widely heterogeneous in terms of scope, between limited single-turn interaction and open-ended multi-session interaction, but different benchmarks have not established standardized

protocols to measure the fundamental agent properties of sample efficiency, task distribution generalization, cost-effectiveness in computational terms, and characterization of failure mode Du and Ding (2021). This heterogeneity makes it difficult to have useful cross study comparisons and clouds underlying questions related to what effective agentic behavior should entail.

There are still critical gaps in the existing assessment environment that inhibit scientific knowledge and real-world application of agentic systems. To begin with, there exist no standardized taxonomies to classify agent capabilities, environmental properties, and assessment dimensions, which makes it impossible to organize knowledge in terms of research activities. Second, a majority of benchmarks focus on the task completion rates and overlook such crucial dimensions as being able to scale to real-world complexity, be consistent with human preferences, safety assurances, and reproducibility in multiple implementations Lebovitz et al. (2021). Third, the field does not have detailed restrictions on methods to assess trade-offs between cost-efficiency, because the agents with greater success rates tend to be orders of magnitude more computational or API calls than other agents. Fourth, rapid advances in foundation models and prompting have made benchmark validity a moving target. Apparent performance gains may reflect dataset contamination or overfitting rather than genuine capability improvements Evtikhiev et al. (2023). In addition, automated test generation, and evaluation techniques do not have the ability to establish whether the generated tests adequately test the intended functionality or are just superficially measured tests (Dakhel et al. 2024; Arabiat et al. 2023). All of these constraints weaken the validity of agent assessment as a science, as well as prevent principled system construction

Although recent surveys have studied the evaluation of LLM Chang et al. (2024) and software engineering applications, e.g., with a survey of use in software engineering, we find no studies that systematically examined the failure to predict deployment success across agent domains with benchmark performance (Wang et al. 2024b; Hou et al. 2024; Martínez-Fernández et al. 2022). The previous work lists the means of evaluation or explains the possibilities but does not address the critical analysis of methodological sufficiency to measure the autonomous decision-making. There are three fundamental disconnects, namely, (1) between measures of task success and process quality (coherence of planning tools) (2) between benchmarking performance and cost-efficiency of deployment; and (3) between an isolated task performance and end-to-end workflow integration. This review is unique in its positioning of evaluation methodology, but not model capability, as the principal constraint to reliable agent deployment.

It is a systematic review of 15 large benchmarks (AgentBench, WebArena, SWE-bench, PaperBench, MLGym, BrowserGym, HumanEval, MBPP, GAIA, ToolBench, TerminalBench, Mind2Web, ALFWorld, BabyAI, HotPotQA). We do not give the description of the methods of evaluation; in contrast to the current surveys which present the description of the evaluation methods, we critically examine one of the reasons why the current practices do not capture the dimensions which are critical in deployment. Table 1 in locates this review in comparison with current surveys.

The main reason as to why such a thorough review is required is the dire necessity to unify, critique and integrate the patchwork quilt of agent evaluation methodologies. With organizations investing a lot in the creation and implementation of agentic systems, lack of stringent and standardized evaluation practices is a threat to technological advancement as well as stakeholder confidence. In addition, the prevailing broadcasting of incomparable

Table 1 Comparison with Existing Surveys on Agent Evaluation

Survey Description	Year	Focus	Domain Coverage	Evaluation Critique	Deployment Gap Analysis	Key Limitation	Our Distinction
Recent Agent Surveys (Mohammadi et al. 2025; Akshathala et al. 2025)	2025	Agent capabilities & SWE-bench critique	Multi-domain	Descriptive	Partial	Lacks quantitative gap scoring framework	We provide deployment-gap rubric (Table 10) with quantitative scoring
LLM Evaluation Methods Survey Chang et al. (2024)	2024	LLM evaluation methods	General NLP	Descriptive	Not addressed	Static tasks only	–
LLM for Software Engineering Survey Hou et al. (2024)	2024	LLMs for SE	Software only	Capability-focused	Minimal	Domain-specific	–
Software Testing with LLM Survey Wang et al. (2024b)	2024	Software testing	Testing/QA	Secondary	Implicit	Narrow scope	–
AI-Based Systems SE Survey Martínez-Fernández et al. (2022)	2022	SE for AI systems	Software	Development practices	Not addressed	Pre-dates agent benchmarks	–
This Review	2026	Agent evaluation methodology	Multi-domain	Systematic inadequacy analysis	Central thesis	N/A	Systematic inadequacy analysis with actionable measurement protocols

standards and uneven reporting criteria does not allow the research community to arrive at common conceptualizations of agent capabilities, limitations, and failure modes. This review will create a precedent to more principled agent assessment practices that can keep up with the rapid technological progress by offering an in-depth examination of the current evaluation frameworks and the critical gaps in their methodology description to address the gap in the current state of the art in the community of practice and make chances to revolutionize the current methodology and practice of agent evaluation as a whole in the future (Wang et al. 2024a; Hou et al. 2024).

This review makes the following contributions to the field of agentic AI evaluation:

- i. An evaluation-inadequacy taxonomy mapping 15 benchmarks to deployment failure modes with quantitative gap scoring, distinct from capability taxonomies.
- ii. A critical analysis demonstrating how test-passing metrics fail to capture deployment-critical dimensions (maintainability, security, cost-efficiency).
- iii. A trajectory-level evaluation framework with explicit cost-reproducibility-validity trade-offs.
- iv. A systematic identification of metric insufficiencies including absence of cost-aware and safety-aware scoring.
- v. A synthesis of emerging evaluation paradigms with barriers to adoption.

Software development is a specific field of application of agents on which this paper pays specific attention in the context of evaluating agents. Coding agents are an ideal example of the core problems with agentic behavior evaluation, since they are required to balance between the low-fidel test-passing measurements with comprehensive code quality features such as maintainability, security, efficiency, and compatibility with changing requirements. Although the early analysis of completion benchmarks focused on the single function of a software agent, modern software engineering agents work throughout the entire development life cycle and need evaluation systems to reflect the end-to-end functionality of a software agent, such as requirements analysis, deployment, and maintenance. The conflict between automated measurements and the correctness of humans in software situations sheds light on the general issue of judging agents in all fields.

The remainder of this paper is organized as follows: Sect. 2 revives historical and technical background agentic AI architectures, core evaluation concepts; Sect. 3 presents our systematic methodology of selecting and analysing literature; Sect. 4 surveys major categories of benchmark and evaluation frameworks; Sect. 5 analyses evaluation dimensions and metrics; Sect. 6 finds applications; Sect. 7 explores methodological limitations and open challenges; Sect. 8 future directs evaluation applications; and Sect. 9 concludes with recommendations on future research directions.

The evolution of AI-evaluation conceptual framework as shown in figure 1 is that the traditional AI-evaluation based models are being replaced by agent-based assessment models as systems become increasingly more autonomous with the ability to interact with the environment.

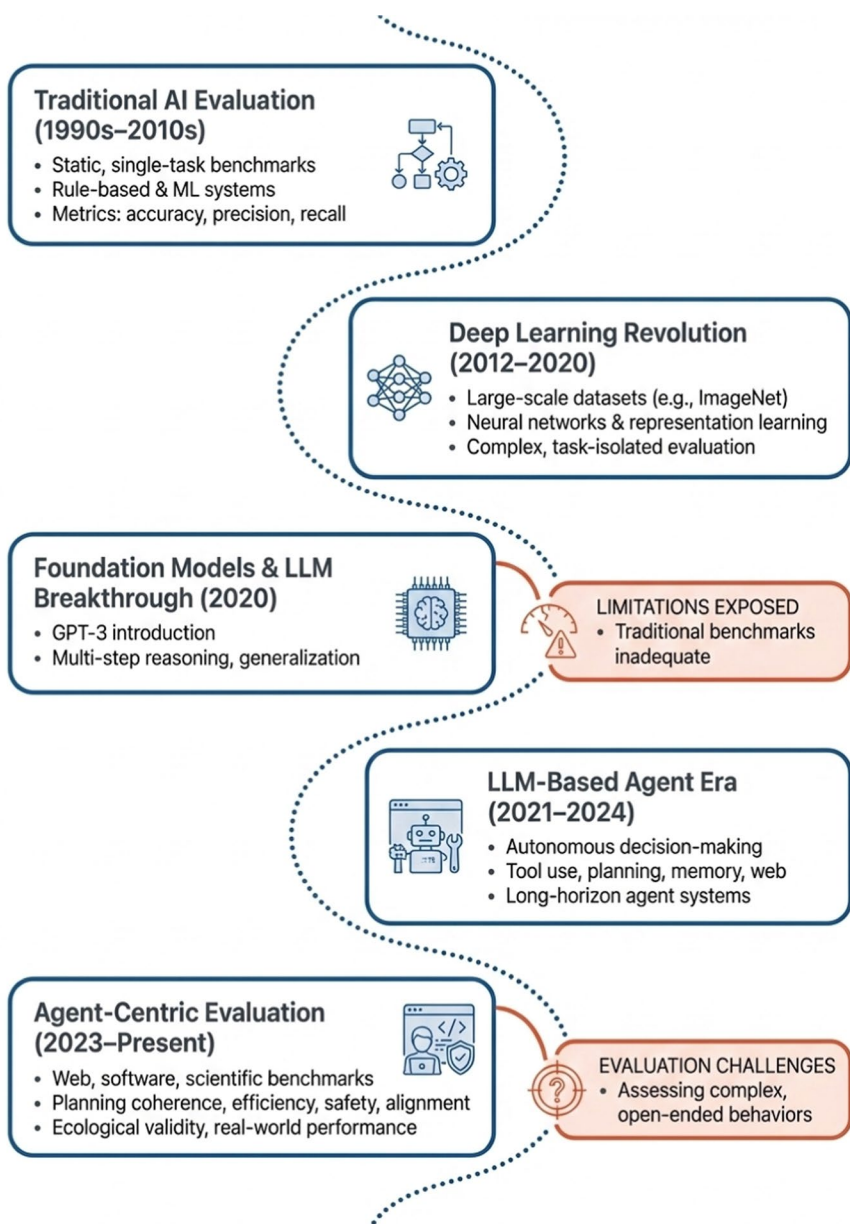


Fig. 1 Evolution of AI Evaluation Paradigms and Agent-Centric Benchmarking

2 Historical and technical background

This part presents the history of AI evaluation paradigms, starting with the concept of a static task-based evaluation and moving to the agent-based evaluation benchmarking framework, defining the theoretical and technical basis required to comprehend the existing evaluation

issues and technological advancements in methods.

2.1 Historical background

The development of artificial intelligence evaluation has been followed by four paradigmatic shifts, each of which mirrors the fundamental changes in the nature of AI systems and their connection to the computational environments. Rule-based and symbolic methods predominated the early AI evaluation period (before 2010) with benchmarks concentrating on discrete reasoning problems with deterministic results Du and Ding (2021). The early AI evaluation era (before 2010) was where benchmarks were in the such form of discrete reasoning problems with deterministic answers. The metrics of evaluation were focused on accuracy and precision in closed world assumptions where system logic did not depend on the dynamics of the environment. This paradigm fitted quite well with experts systems and symbolic planners, in which success could be decided by logical correctness.

In 2010-2017, the statistical and deep learning revolution offered large-scale supervised benchmarks that were focused on optimizing single-task performance. ImageNet paradigm created evaluation standards based on fixed datasets, held-out test sets, and scalar evaluation metrics Lebovitz et al. (2021). This method was very effective in assimilating perceptual activities and showed underlying constraints in evaluating higher-order cognitive processes, including reasoning and adaptive decision-making Wang et al. (2024b). The benchmarks of this period viewed AI systems as passive functions approximators and quantified what models could predict as opposed to what they could do by sequential action. As systems were unable to cope with distribution shift and task generalization, the difference between the performance in laboratories and in the real-world grew larger and larger Nashaat and Miller (2024).

With the advent of foundation models and large language models (2018-2021), the conventional evaluation schemes were broken because they introduced systems that can learn in few shots, instruct, and use crude tools in a rudimentary fashion. The pretraining-finetuning paradigm allowed models to be able to learn generic skills that can be used in a variety of tasks Wang et al. (2024a). Pretraining-finetuning paradigm enabled models to have general skills which can be used in a wide range of tasks Dong et al. (2024). Initial studies found that Chain-of-thought reasoning and other forms of emergent behavior that were not directly optimised during training were observed in the LLMs, and were not well captured by existing metrics Munikoti et al. (2023). The problem with the use of the static evaluation sets emerged as the evidence was accrued of the contamination of benchmarks by the use of web-scale pretraining data Evtikhiev et al. (2023).

The subsequent emergence of agentic AI (2021-present) has required the very conceptualization of the evaluation approach to be revisited because AI systems have ceased to serve as passive predictors and have become autonomous decision-making agents, functioning in the complex dynamic environments Wang et al. (2024a). Modern agents have long-term strategic planning, long-lasting memory between agents, can use external tools and APIs, and respond to environmental feedback by adjusting their behavior Nair et al. (2023). This change revealed the deficiency of accuracy-based measures because agent performance lies in the completion of tasks cumulatively, efficiency of resources, and correspondence to human wills in multi-step paths and trajectories, among others, as the human wills are inadequately pronounced in accuracy-based metrics Dakhel et al. (2024). Agents were tested in

more realistic task environments by specialised benchmarks like AgentBench, WebArena, SWE-bench, MLGym, PaperBench, and BrowserGym, which have been published since the year 2020 and prior to 2020, respectively (Liu et al. 2023; Hou et al. 2024; Jimenez et al. 2023). Fig. 2 shows the evolution of AI evaluation paradigms from static task-based benchmarksto interactive agent-centric evaluation frameworks.

2.2 Technical background

There are five key capabilities that single out agentic AI systems as distinct and separate as compared to traditional machine learning models. Long-horizon reasoning and planning help agents to break down complex goals into executable sub-goals and have consistent strategies through hundreds of steps of interaction in a coherent way Mohan et al. (2024). The use of tools and APIs enables agents to expand their functionality by calling on external computational services, databases, and specialty models, which requires evaluation of tool selection fidelity and a high level of integration of tool outputs to be productive Munikoti et al. (2023).

This review concentrates on LLM-based agentic AI systems where language models serve as the core reasoning engine for multi-step decision-making, tool selection, and environmental interaction. The evaluated benchmarks (Sect. 4) predominantly assess this architecture. While reinforcement learning-based agents (e.g., game-playing AI, robotic control) and hybrid neuro-symbolic systems exist, they employ fundamentally different evaluation paradigms reward optimization and logical reasoning verification, respectively which fall outside our systematic analysis. Our findings regarding metric inadequacy, cost blindness,

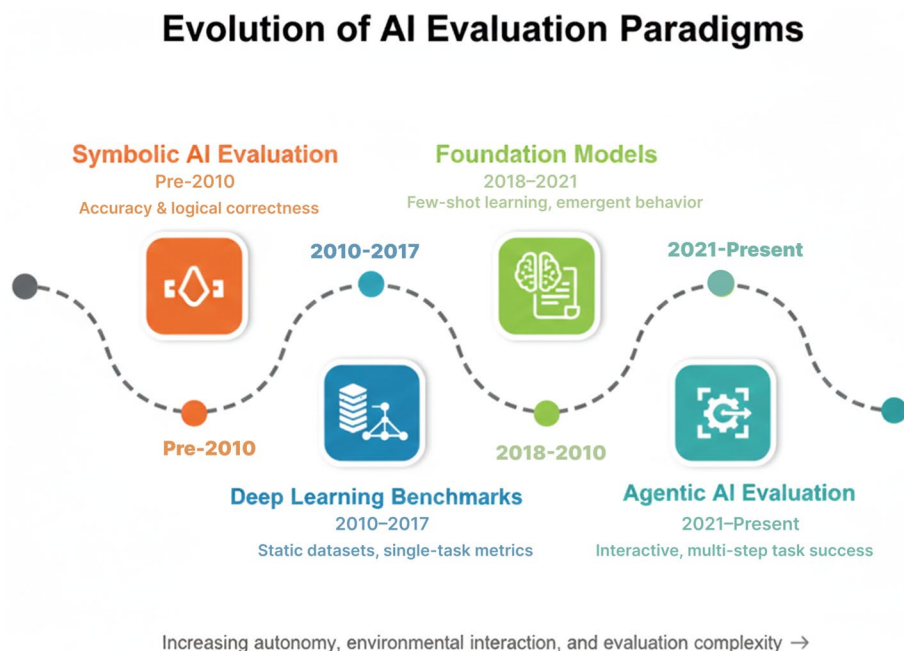


Fig. 2 Evolution of AI evaluation paradigms from static task-based benchmarks to interactive agent-centric evaluation frameworks

and trajectory evaluation needs apply specifically to LLM-based agents, though some principles may generalize to other paradigms.

Memory systems include short-term working memory, long-term episodic storage and external knowledge bases that allow agents to have context in real-time, over a prolonged interaction with each other Wang et al. (2024a). The interaction with the environment functions enable agents to get states, do actions, and receive the feedback in web interfaces, code repositories, and simulators Zhang and Mo (2021). Self-reflection and adaptation allow the agents to assess their performance and find mistakes as well as change strategies without outside interference Nair et al. (2023).

The methodologies of agent evaluation have key fundamental differences with the classic machine learning assessment on several fundamental dimensions. The measure of the performance of agents using static evaluations on fixed tests with known correct answers provides reproducibility, but it lacks ecological validity because it uses fixed tests with predetermined correct answers only Terragni et al. (2025). Interactive evaluation solves the problem by embedding agents in dynamic systems in which actions change state and future observations, which offer realistic assessment at the price of the high variance and cost Yan et al. (2025). ask success measures final results, which do not indicate the quality of decisions or efficiency of resources made in the middle of the task, which means that the metrics fail to address intermediate results and efficiency of the activity Munikoti et al. (2023).

Trajectory based evaluation provides complete plan sequences, evaluation of plan coherence and error recovery, but needs more complex evaluation infrastructure Mohan et al. (2024). Minimal feedback in the form of rare reward signals only when there is terminal success or failure hinders credit assignment, making this difficult to achieve in deep learning models Nguyen et al. (2020). Automated metrics can be used to do a scalable assessment by programmatic verification but might be incapable of representing nuanced aspects of quality Evtikhiev et al. (2023). Human-in-the-loop evaluation includes the subjective qualities of human judgment, which uses gold-standard evaluation at prohibitive cost Chen et al. (2024).

The agentBench offers sophisticated assessment in various domains of tasks such as web shopping and database querying and generalization in heterogeneous settings are measured with an evaluation of the agentBench across all of them, taking into account the variety of the environment in question Liu et al. (2023). WebArena is built as real world web settings that reflect the web sites of e-commerce and social media with a particular focus on ecological validity with authentic interface complexity Thakkar et al. (2025). SWE-bench assesses software engineering agents with values to genuine GitHub issues about the production repositories and quantifies the ability to comprehend codebases and provide functionality Jimenez et al. (2023). PaperBench assesses agents on scientific literature analysis tasks including information extraction and hypothesis generation Starace et al. (2025). MLGym offers the standardized environments to automate the machine learning workflow automation Kamaloo et al. (2025). BrowserGym provides the interactive web navigation testing using the automation of the browser of web navigation evaluation (WebChoreArena 2025). Figure 3 Comparison between traditional static model evaluation and agent-centric evaluation paradigms, highlighting trade-offs in realism, cost, reproducibility, and safety.

STATIC VS. AGENT-CENTRIC EVALUATION PARADIGMS

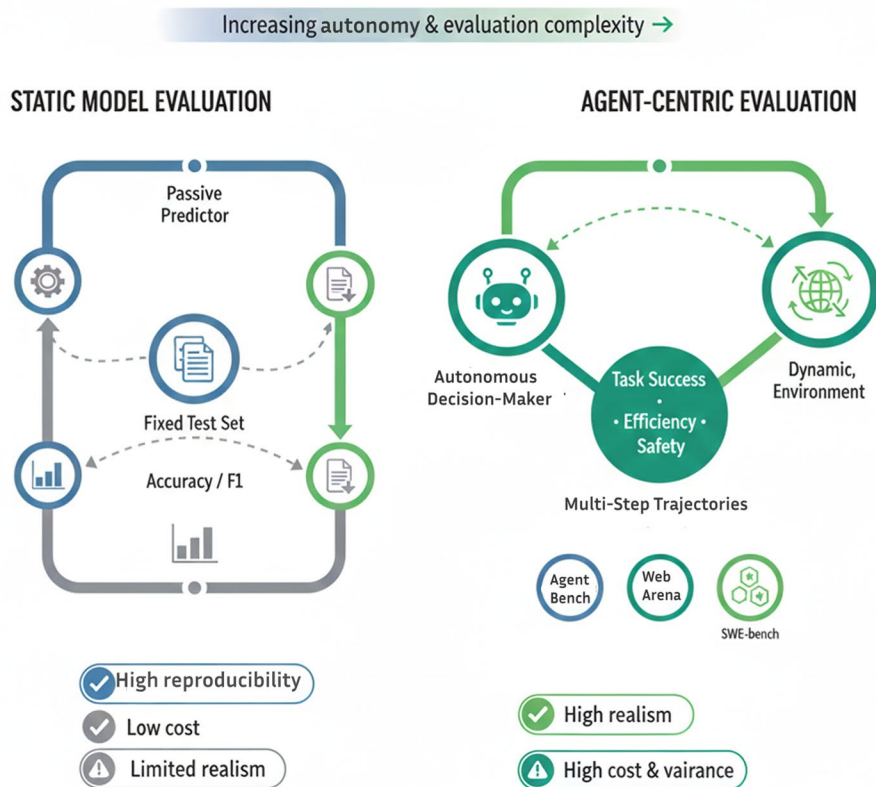


Fig. 3 Comparison between traditional static model evaluation and agent-centric evaluation paradigms

2.3 Relationship between historical and technical evolution

The historical development of the AI ability and the assessment modes shows certain patterns according to which evaluation systems are always behind the technological one. Fixed benchmarks matched well with the initial AI systems as stateless functions. Terragni et al. (2025). With the acquisition of adaptive behavior by the systems based on machine learning, the evaluation changed towards probabilistic evaluation, although the assumption of passive model operation persisted. This paradigm is disrupted by the shift to agentic AI system and needs evaluation at the environment level as well of course Yan et al. (2025).

Greater realism in evaluation brings about harsh trade-offs. Simulated task settings improve ecological validity at great computer cost. The process of benchmark saturation accelerates when competent agents learn to do prevailing tasks, thus requiring the constant creation of benchmarks Lebovitz et al. (2021). Agents are prone to increased safety risks when they are put in real-life circumstances with real-life implications. Cross-benchmark

comparability is a problem because different structures use different criteria of success and measurement protocols.

2.4 Challenges and limitations in historical context

Modern agent analysis continues to struggle with the historical issues of agent analysis. The problem of benchmark overfitting and dataset contamination has been exacerbated by foundation models being trained on web-scale corpora which may contain benchmark data Evtikhiev et al. (2023). The lack of a shared evaluation taxonomies renders formal categorization of agent capabilities and evaluation procedures. Measurement The measurement of generalization and robustness is still unsatisfactory, since benchmarks generally measure how well prediction is on particular task distributions without describing how such prediction will degrade with a change in the distribution of the test Nashaat and Miller (2024).

The issue of safety and alignment is not given appropriate attention in the agent benchmarks which only ensure that the tasks are done and they do not measure harmful actions avoidance appropriately. Realistic agent evaluation has high computational and financial costs that cause barriers to accessibility Chen et al. (2024). Environment nondeterminism and lack of proper documentation of evaluation protocols are some of the challenges that face reproducibility. Past tendencies show that the evaluation systems are always lagging behind the development of capabilities. This shortcoming encourages swift creation of ongoing, adaptive, and integrated agent assessment systems that can quantify new abilities and identify safety dangers Dakhel et al. (2024).

3 Systematic review methodology and evaluation techniques

Assessment of agentic AI systems poses methodological challenges never seen before that place it at the core of what is different about the assessment of traditional machine learning. The agentic AI system have sequential decision-making, tool invocation, environmental interaction, and adaptive planning over long periods of time as compared to the static models which give rise to single outputs. It is a systematic analysis of the methods of evaluation that explicitly target agentic systems and presents the methodology as a logical taxonomy that includes the basic benchmarking techniques, interactive assessment procedures, and domain-specific techniques of evaluation. We focus on software development agents (as a main case study), illustrating how the evaluation issues arise in the context of application, and how the issue of architectural decisions has an impact on the strategy of assessment.

3.1 Benchmark selection and evaluation protocol

To ensure reproducibility and transparency, we establish explicit criteria for benchmark selection and our evaluation framework. Our 15 benchmarks were selected based on four inclusion criteria. First, temporal relevance was ensured by focusing on publications between 2021–2025 to capture the LLM-based agent era, consistent with the PRISMA search window reported in Section 3.2. Second, all selected benchmarks require multi-step interaction involving sequential decision-making beyond single-turn responses. Third, community adoption was verified through citation counts (≥ 50 citations) or publication in major ML

venues including NeurIPS, ICLR, ICML, ACL, and EMNLP. Fourth, we ensured domain coverage by selecting benchmarks representing diverse application areas such as software development, web navigation, embodied AI, scientific reasoning, and general question answering.

We explicitly excluded single-turn QA benchmarks without environmental interaction (e.g., MMLU, HellaSwag), domain-specific benchmarks without generalizable evaluation patterns (e.g., medical diagnosis-only systems), and closed-source proprietary benchmarks without public documentation. Each benchmark was systematically analyzed across six evaluation dimensions. Task structure analysis examined horizon length (categorized as short for less than 5 steps, medium for 5–20 steps, and long for more than 20 steps), interaction type (static versus interactive), and environment determinism. Metrics coverage analysis identified what each benchmark measures (such as success rate, efficiency, and cost) and critically, what dimensions are systematically omitted (including safety, robustness, and process quality). Dataset properties were assessed for size, contamination risk, and trajectory annotation depth. Evaluation validity was examined through correlation with deployment outcomes where empirical data exists. Reproducibility was assessed based on environment stability, documentation quality, and public availability. Finally, deployment gaps were identified by documenting mismatches between benchmark performance and real-world requirements.

Rather than applying numerical weighting, which would impose artificial commensurability across heterogeneous benchmarks, we employ comparative inadequacy analysis. For each benchmark, we identify which deployment-critical dimensions are systematically absent from evaluation protocols. This qualitative-comparative approach is appropriate given that benchmarks measure fundamentally different capabilities (code generation versus web navigation versus embodied control) and cannot be meaningfully ranked on a single numerical scale. Our contribution lies in revealing structural patterns of evaluation inadequacy across domains rather than scoring individual benchmarks. Tables 2, 3, and 5 document our systematic application of these evaluation dimensions to each benchmark, enabling independent verification of our analysis.

To assess rubric reliability, two authors independently applied the gap-scoring protocol (10) to five benchmarks (AgentBench, SWE-bench, WebArena, HumanEval, and ALFWorld) across all five scoring dimensions (25 ratings per annotator, 50 total). Exact agreement was achieved on 21 of 25 dimension-benchmark pairs (84%), with the remaining 4 cases differing by one point on the 0–5 scale. Disagreements were resolved through structured discussion before scores were finalized for all 15 benchmarks.

3.2 Systematic review protocol

To ensure transparency and reproducibility, we conducted a systematic literature search following PRISMA guidelines, employing explicit search strategies, inclusion/exclusion criteria, and quality assessment protocols to identify and analyze the 15 primary agent benchmarks examined in this review.

3.2.1 Search strategy

Our systematic search was conducted across five major databases:

Table 2 Comparison of Agentic AI Evaluation Techniques

Evaluation Method	Interaction & Horizon	Cost & Reproducibility	Validity & Examples
Static Task Benchmarks	Non-interactive, single-step	Low cost, high reproducibility	Low realism; GALA, MMLU
Interactive Environment Evaluation	Stateful, multi-step	High cost, medium reproducibility	High realism; WebArena, AgentBench
Domain-Specific (Software Development)	Stateful, task-driven	Medium-high cost, medium reproducibility	Very high validity; SWE-bench, Terminal-Bench
Hybrid / Continuous Evaluation	Stateful, longitudinal	Very high cost, low reproducibility	Highest realism; production deployment

- IEEE Xplore
- ACM Digital Library
- arXiv
- Google Scholar
- Springer

The search period covered January 2021 to December 2025, corresponding to the emergence and rapid development of LLM-based agentic systems as documented in our historical background. We employed three complementary search string strategies. The primary search string combined (“agentic AI” OR “AI agent” OR “autonomous agent”) with (“evaluation” OR “benchmark” OR “assessment”). Our secondary search string targeted (“LLM agent” OR “language agent”) paired with (“testing” OR “validation” OR “metrics”). Domain-specific searches used (“software agent” OR “web agent” OR “code generation”) combined with (“evaluation framework” OR “performance metric”). These strings were adapted to each database’s syntax requirements while maintaining semantic equivalence.

The initial database search identified 1,247 papers. After removing duplicates and conducting title and abstract screening, 384 papers remained for detailed review. We applied explicit inclusion criteria requiring that papers must propose or evaluate agent benchmarks, demonstrate multi-step autonomous behavior assessment, be peer-reviewed or high-impact preprints (arXiv papers with more than 50 citations), and provide sufficient evaluation methodology details. Exclusion criteria eliminated papers focusing solely on single-task static ML models, purely theoretical work without empirical evaluation, narrowly domain-specific approaches without generalizable insights, and duplicate publications of the same benchmark. Full-text review of the 384 papers against these criteria resulted in 95 papers proceeding to eligibility assessment.

From the 89 eligible papers, 52 papers met all inclusion criteria through detailed quality assessment. These 52 papers include the 15 primary benchmarks analyzed in depth throughout this review, plus 37 supporting works on evaluation methodology, metrics design, and deployment studies that inform our critical analysis. The 15 benchmarks were selected as representative cases based on four criteria: citation impact (more than 100 citations or evidence of major deployment), methodological diversity in evaluation approaches, domain coverage spanning software development, web automation, embodied agents, and research automation, and public availability of evaluation protocols enabling reproducibility assessment. Figure 4 presents the complete PRISMA flow diagram documenting this selection process.

For each of the 15 benchmarks, we systematically extracted five categories of information: evaluation methodology (static versus interactive, single-turn versus multi-step), primary metrics used and their mathematical formulations, dataset characteristics including size, task diversity, and trajectory depth, task domains and application areas, and reported limitations or deployment considerations when available. This structured extraction protocol forms the empirical foundation for our comparative analyses presented in Tables 2, 3, 5, 3, and 9. We assessed benchmark quality across four dimensions: reproducibility based on protocol documentation completeness, ecological validity reflecting real-world task alignment, metric comprehensiveness covering capability, cost, safety, and robustness dimensions, and explicit consideration of safety or alignment concerns. This quality assessment

Table 3 Representative Tools and Frameworks for Agentic AI Evaluation

Framework	Evaluation Domain	Interaction Type	Primary Metrics / Focus
AgentBench Liu et al. (2023)	Multi-domain agent tasks	Interactive, stateful	Task success rate, cross-domain generalization
WebArena Thakkar et al. (2025)	Web navigation and automation	Interactive, browser-based	Functional correctness, task completion
SWE-bench Jimenez et al. (2023)	Software engineering (bug fixing)	Repository-based, executable	Test pass rate, issue resolution
PaperBench Starace et al. (2025)	Research automation	Multi-step, tool-assisted	Result reproduction accuracy, rubric score
BrowserGym WebChoreArena 2025	Web-based agent environments	Interactive, standardized	Reproducibility, environment coverage
Terminal-Bench Gulli (2025)	Command-line proficiency	Shell-based interaction	Task success, execution correctness
GALA Mialon et al. (2023)	Real-world QA with tools	Multi-step reasoning	Answer correctness, tool use validity
ToolBench / API-Bank Li et al. (2023)	Tool and API usage	Nested function calls	Call accuracy, argument correctness

directly informed our critical analyses in Sects. 5 and 7, where we identify systematic gaps between benchmark evaluation and deployment requirements.

3.3 Categorization of evaluation techniques

Agentic AI assessment methods can be logically divided into three categories. The first involves static task benchmarks, the foundational approach for evaluating agents on discrete task specifications, where complete solutions are generated without environmental feedback loops Chang et al. (2024). These approaches are based on traditional AI evaluation and are used to assess such capabilities as understanding natural language, retrieving knowledge, and single-step reasoning using preset test sets Xu et al. (2022).

The second type of evaluation is interactive environment based evaluation wherein agents act in simulated or real worlds that give them dynamic observations of the states and the consequences of actions. The given paradigm is consistent with the traditions of reinforcement learning evaluation but generalized to the language-model-based agent exploring the web interfaces, file systems, and API ecosystems Gemp et al. (2022). Multi-agent evaluation systems also create additional complexity in assessment by creating a coordination, competition, and emergent behaviors that cannot be reflected by a static benchmark assessment. Canese et al. (2021).

The third category is domain specific evaluation frameworks where assessment methodologies are developed based on specific application settings. The field of software development is the most developed and there are specialized benchmarks of code generation, debugging, test synthesis and repository understanding Schäfer et al. (2023). These domain-specific methods recognize that general-purpose measures are not sufficient to assess the professional competence needs, requiring task-specific success measures and quality standards to be satisfied in particular tasks, as well as quality requirements and thresholds to be met in specific tasks Shafiq et al. (2021).

3.4 Explanation of key methods

In this sub-section, the basic technical descriptions of the core methodologies of evaluation that are applied to agentic AI assessment will be described in detail: task-based success evaluation, interactive environment protocols, and code execution verification. Figure 5 demonstrates the agentic AI review loop, which includes goal definition, action implementation, feedback, and scoring the outcomes.

3.4.1 Task-based success evaluation

Task-based evaluation works based on episodic judgment cycles in which agents are provided with natural language task descriptions, with the agents then generating action sequences or direct outputs, as well as success judging based on predefined criteria. The evaluation process starts with sampling tasks of curated distributions, agent execution with optional multi-turn interaction, and automated analysis of the quality of the generated outcomes, or manual evaluation of quality.

Mathematically, this algorithmically calculates success rates by dividing the tasks that pass by the total attempts with variations in the form of pass-at-k that characterize multiple

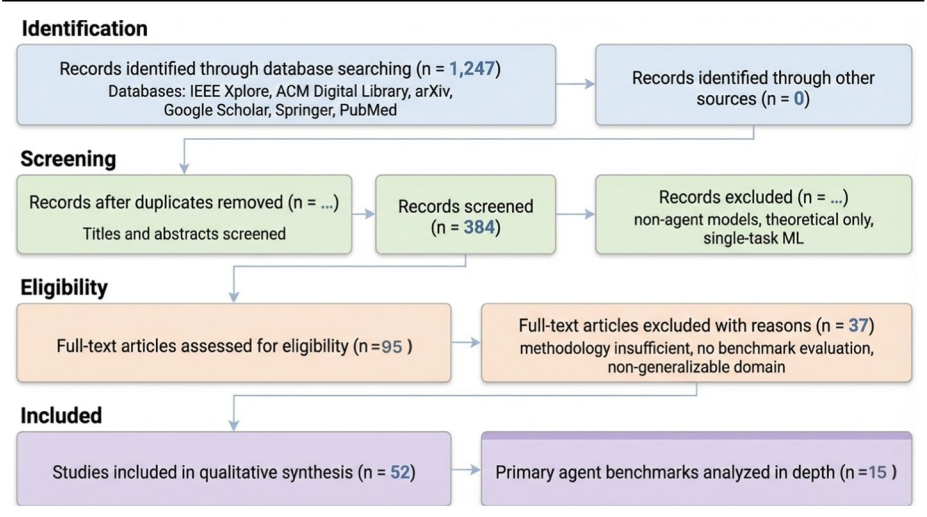


Fig. 4 PRISMA Flow Diagram: Systematic literature search and selection process for the 15 primary agent benchmarks analyzed in this review

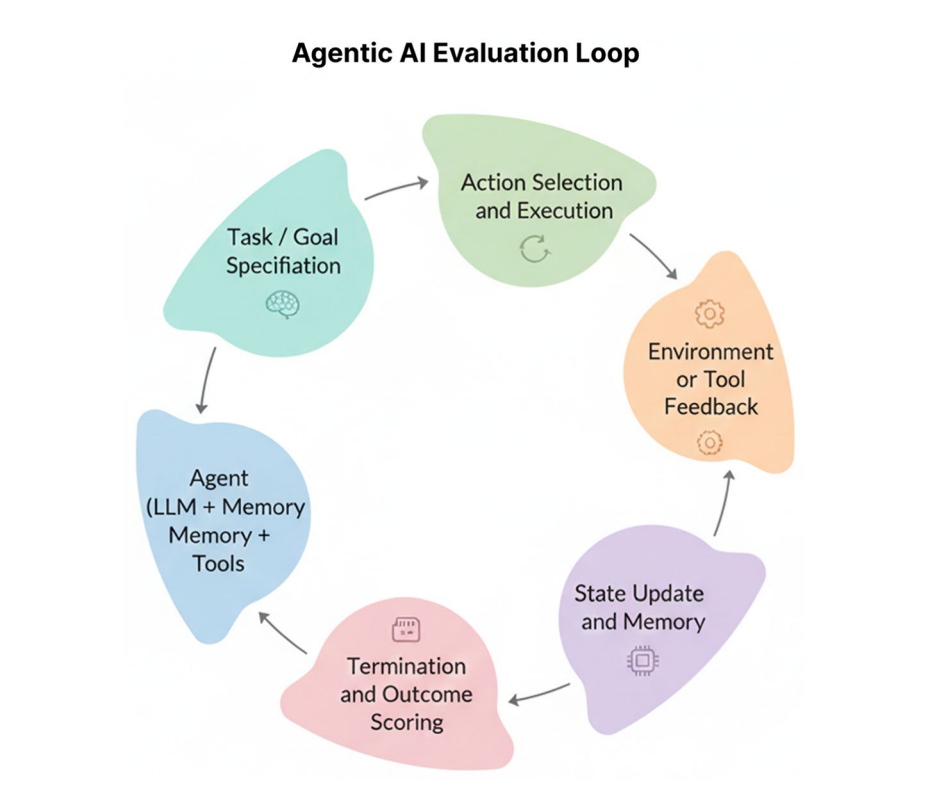


Fig. 5 Agentic AI evaluation loop

independent trials to estimate the probability of success under sampling variability Yeo et al. (2024). The dimensions of evaluation include the ability to follow instruction, accuracy of knowledge retrieval as gauged by the ability to answer multi-hop questions, and the ability to withstand variations in task phrasing as evaluated by multi-hop question answering Chen et al. (2025).

The main strength is efficiency and interpretability of evaluation allowing quick assessment of capability in a wide range of categories of skills. Nonetheless, such critical limitations as benchmark contamination in which training data spills the solutions to tasks, overfitting to small task distributions that cannot be transferred to related problems, and failure to evaluate long-horizon planning or adaptive behavior exist Anwar and Bashir (2023).

3.4.2 Interactive environment evaluation

Interactive evaluation inserts the agents into the stateful environments to facilitate the observation-action loops over several timesteps. The protocol sets environment state, gives agents initial observations and available action specification, run agent selected actions to update environment state, and repeats until terminal conditions or time limits.

The algorithmic basis is based on trajectory rollouts in which action sequences are generated by agents given sequence of observation histories. Success computation evaluates final environment states against goal specifications, with intermediate rewards optionally accumulated across trajectories to assess efficiency alongside correctness Hu et al. (2024). Assessed skills are the choice of tools and turn-taking contextual memory, the ability to retain information when failing to execute an action, and adaptive planning in response to the ineffectiveness of earlier plans Sivamayil et al. (2023).

Realistic evaluation of agent autonomy, identifying failure modes that are not evident in single-turn evaluation, and quantifying actual operational utility in task completion are all considered strengths of the approach to artificial intelligence in tasks completion Rahman et al. (2022). The drawbacks include, but are not limited to, high computational needs to sample a trajectory, the impossibility of performing reproducibility due to non-determinism in the environment, and the inability to attribute failures on a case-by-case basis to a particular set of capabilities being insufficient in a complex multi-step problem Yu et al. (2021).

3.4.3 Software development evaluation protocols

The analysis of code generation agents can not be evaluated by means of executable verification other than the similarity of languages. The evaluation process is performed in the following stages: problem specification presentation, agent code generation, sandboxed execution in isolated environment and test suite execution to ensure the functional correctness. The scoring algorithm calculates the pass-at-k measure on several code samples, the likelihood that at least one solution generated meets all test cases produced by the algorithmic methodology Dong et al. (2025). Long evaluation dimensions measure the quality of the code by use of the static evaluation by identifying any security weaknesses, the measure of efficiency by examining the runtime and memory diagnostic, the measure of maintainability by evaluating the complexity of the code, and the measure of adherence to style guidelines by evaluating the style conventions of the programmatic code or the code itself in style analysis and metrics of complexity, and style constraints Guo et al. (2023).

Such a methodology is a direct measure of operational utility which can be used as actionable feedback to improve iteratively. But the incompleteness of the test suites introduces gaps in the evaluation process by passing tests not ensuring that they are correct on unobserved inputs. There are other challenges such as measuring the understanding of codes to debugging tasks and measuring competence of an end-to-end software development life-cycle's competence other than the isolated function generation.

3.5 Hybrid and emerging methods

The new forms of evaluation are multidimensional assessment methodologies that integrate various approaches of evaluation to address the shortcomings of each method. Continuous benchmarking deploys continuous reviewing since agents change, monitoring performance stability, and identifying capability deterioration over time of deployment of an agent Martínez-Fernández et al. (2022). This longitudinal design helps to show time-based dynamics, which snapshots cannot show. Human-in-the-loop assessment incorporates the use of expert opinion on dimensions that do not lend themselves to automated quantification, such as the readability of the code, the quality of explanations and the correspondence to unarticulated user goals. Human evaluation increases validity, but creates a limitation of scalability and inter-rater reliability issues.

Multi-agent competitive evaluation uses agents based on the game-theoretic settings where comparative performance against adversarial or cooperative agents can be used to provide a better signal of capability as compared to absolute measures of performance. Moreno and Bitterman (2024). The future of evaluation lies in the environment-grounded evaluation where the evaluation is measured by the fact that the intervention is developed and deployed to a real production environment, with such metrics as the user acceptance rate, intervention frequencies, and outcome quality.

3.6 Comparative analysis of techniques

The choice of evaluation technique must be done through an in-depth tradeoff evaluation in various dimensions. The benefit of using the static benchmarks is that it offers the high capability coverage of discrete skills but does not offer much insight into emergent agent behaviors. The interactive environments provide a better measurement of generalization by identifying the existence of overfitting by instantiating the environments in a diverse manner, but the static benchmarks are prone to memorizing as opposed to reasoning (Fig. 6).

Computational efficiency is extremely biased towards static evaluation, where benchmark run times are measured in minutes compared to hours of interaction with the rest of the environment. Scalability aspects also favor the use of the static approach of assessing large model populations but the interactive protocols allow a more in depth analysis of a single system.

Empirical validity is positively correlated to evaluation realism, indicating that a hybrid method combining effective screening in the form of a static model with specific interactive models should be considered as one of the best ways to manage resource allocation Huang et al. (2023). These dimensions would be compared in a framework of tabular form: static benchmarks are efficient and effective; interactive settings are realistic and offer evaluation

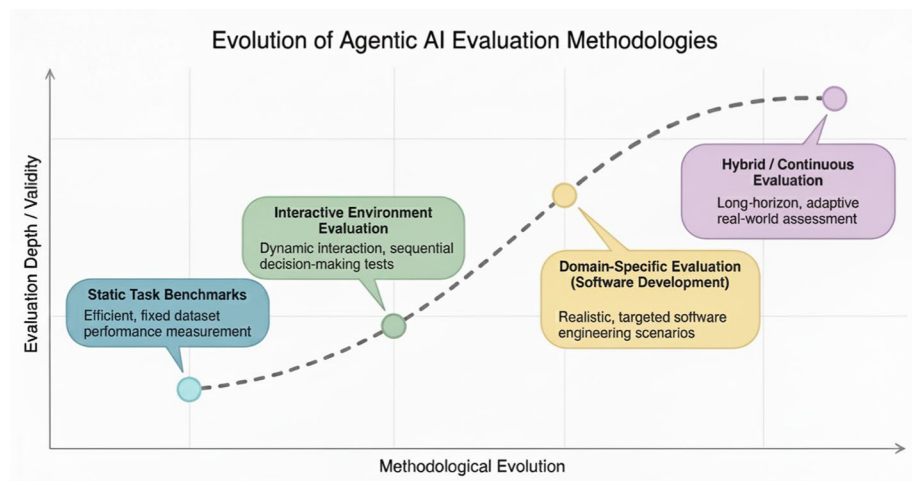


Fig. 6 Evolution of Agentic AI Evaluation Methodologies

of generalization; methods specific to domains are valid to specialized usages; methods combinative of strategic methods offer tradeoffs.

3.7 Tools, frameworks, and practical implementation

This subsection provides an overview of the main tools/benchmark frameworks applied to assess agentic AI systems, their practical design decisions, area of evaluation and their practical implementation in real-world evaluation environments. Table 3 identifies the key tools and benchmark frameworks in agentic AI system evaluation.

The multi-domain assessment of the eight interactive tasks of AgentBench such as operating system navigation, database querying, and question answering that demand knowledge knowledge is evaluated and the versatility of agents is assessed by the aggregation of cross-domain performance Liu et al. (2023). WebArena creates real-world web navigation scenarios using self-hosted websites that have dynamically generated content, shopping carts and form submissions, and evaluates the practicality of web automation features on real world web navigation scenarios Thakkar et al. (2025).

SWE-bench focuses on evaluation of software engineering by actually resolving Github issues, where agents are required to read and understand pre-existing codebases, write bug fixes, and pass repository test suites Jimenez et al. (2023). PaperBench assesses automation in research by asking the agents to replicate machine learning papers based on a description and assessing scientific competence as the accuracy of reproducing the results of those papers found in the literature Starace et al. (2025).

MLGym specializes in machine learning research agents, offering customizable research tasks including algorithms and experimental analysis of them, which can be customized to any degree of choice Kamalloo et al. (2025). Watching over BrowserGym is a collection of web-based assessment frameworks which standardize the environment interfaces and task distributions to reproducible agent assessment WebChoreArena (2025).

GAIA (General AI Assistants) benchmark is a question-answering that involves multi-step reasoning, multimodal processing, and tool use at three levels of difficulty and is tested in the real world under general AI assistants (GAIA) benchmarking Mialon et al. (2023). MINT evaluates the interactions using tools and language feedbacks in multi-turns and assesses the capacity of agents to integrate iterative feedback. ToolBench and API-Bank measure tool-use proficiency by using realistic API engagement scenarios and embedded function calling patterns Li et al. (2023).

Terminal-Bench measures command-line proficiency across shell-based tasks including setup, debugging, and execution operations Gullí (2025). Assistant Bench tests open-web navigation on the time-consuming realistic tasks. Medical reasoning capabilities are evaluated by domain-specific systems, such as MedQA and BioASQ Borazio et al. (2025), financial analysis competence is measured by FinanceBench.

Some of the considerations of practical implementation are infrastructure requirement, interactive structure has high computational demand to support environment; the evaluation time, which involves a long execution time to conduct the comprehensive evaluation; and interpretation of results, which is the distinction between capacity limitations and artifacts of implementation. The choice of a framework is highly contingent on the goals of evaluation, resources available, and capabilities of particular agents that are being studied.

4 Datasets

Datasets form the empirical basis on which agent assessment is based but they are highly designed in that they influence and often limit what is measurable about autonomous behavior. This sub-section is a critical analysis of datasets as evaluation tools, and the distinction between evaluation datasets and training corpora and the implications of dataset properties on the validity and credibility of benchmark claims.

4.1 Agent-centric perspective on datasets

The analysis of agentic AI systems requires a basic re-understanding of the role and structure of datasets. In contrast to the static machine learning models, which apply to independent input–output pairs, agentic AI systems continuously interact with environments, make sequential decisions, call tools, and engage in multi-step reasoning in the future of artificially intelligent systems and machine learning, where all elements are linked Wang et al. (2024a). This requires the separation of the training datasets used to pretrain or fine-tune foundation models and evaluation datasets assembled with the goal of evaluating agent behaviour in realistic conditions. Training datasets build agent capabilities; evaluation datasets assess whether agents apply that knowledge coherently across trajectories involving perception, planning, and action.

The agents evaluation datasets are fundamentally different in form and intent compared to traditional benchmarks of evaluation Shridhar et al. (2020). Instead of assessing agent performance on single tasks, agent assessment involves recording the trajectory data: full streams of state observations, in-between reasoning steps, calls to tools, responses of the environment and corrective measures. These trajectories show not only whether an agent was successful, but also how it solved problems, learned to respond to feedback, and how it

moved around the environmental constraints Deng et al. (2023). In the absence of annotations at the trajectory level, the notion of evaluation becomes limited to outcome scoring and the decision processes that distinguish between capable agents and those that are brittle systems is lost.

The evaluation frameworks that have been discussed in Section 3 are directly limited by this trajectory-centric view. Design of the dataset determines which measures can be calculated (e.g., partial credit should involve fine-grained measures of success), the operationalization of task success (e.g., is it based on execution or a human being makes the decision), and what benchmarks can be used to compare systems across systems Yang et al. (2018). It is impossible to test API reasoning with a dataset without tool usage annotations, adaptability to non-deterministic environments with a dataset that lacks branching state sequences. The soundness of agent benchmarks, therefore, inherently relies on the encoding of the behavioral traces, which would be used to enable rigorous evaluation, in evaluation datasets. As shown in Fig. 7, displays the loop of a trajectory in which the final states are not the only ones that are captured through a trajectory dataset, but all interactions between the agent and the environment.

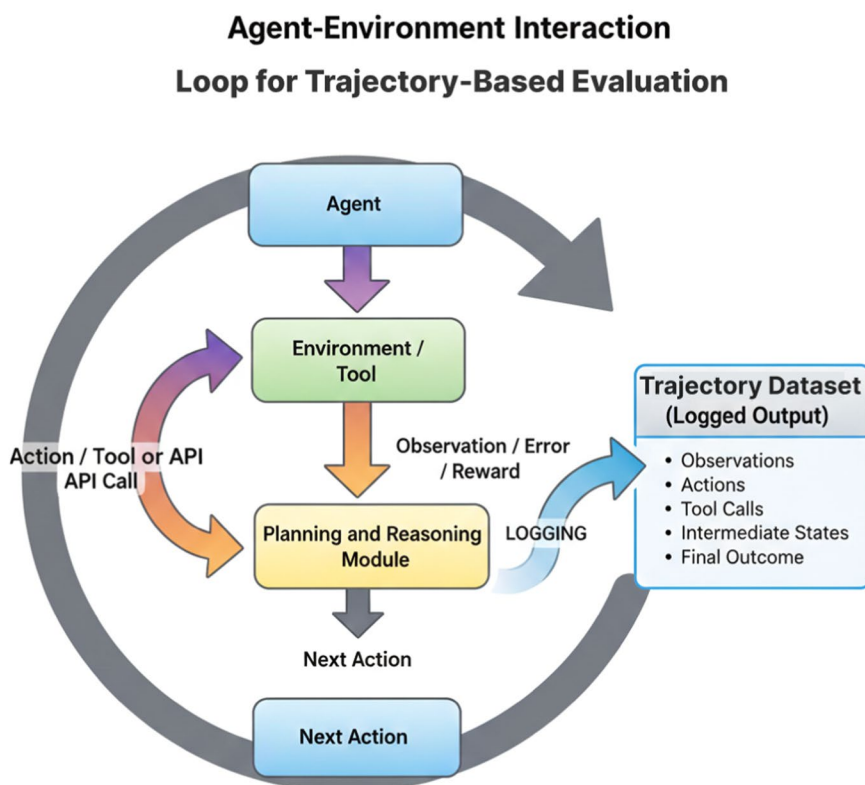


Fig. 7 Agent–environment interaction loop for trajectory-based evaluation. The diagram illustrates how sequential actions, environment feedback, and internal reasoning states are logged to form trajectory datasets for agent assessment

4.2 Dataset typology and core characteristics

To structure agent datasets, it is time to go beyond domain-based classification to a taxonomy that focuses on the evaluation affordances the kind of agent behaviors that each dataset can allow the researcher to estimate. Three major types are known to us: Training Datasets include the corpora in which an LLM is pretrained or where agent policies are fine-tuned to behave successfully on a test set; Benchmark Databases are sets of established tasks with well-structured criteria of success that can be used to assess performance across agents; Trajectory Databases include log history of agent-environment interactions upon which post-hoc inferences can subsequently be performed and an agent can be practiced to behave well.

In these types, a number of features play a critical role in determining evaluation validity. Gundersen et al. (2022). Trajectory length is a measure of time horizon that agents should uphold coherent behaviour short horizons (1-5 steps) evaluate reactive skills, and long horizons (20+ steps) evaluate sustained plan. Interactive complexity describes the presence of either branching state dynamics or adversarial response in the environments. Task organization Multistage task structure The atomic tasks versus compositional goals Multistage task structure is a method used to characterize and differentiate between atomic tasks and compositional goals that must be broken down into a sequence of subgoals. The use of the tools is annotated; it defines whether datasets allow analyzing the agent reasoning concerning external resources or not He et al. (2025).

The quality of annotation is extremely different. Execution-based verification also gives objective success indicators of code tasks and can fail to give semantic correctness related information in code based evaluation Chen (2021). Human judgments present subtle judgments at the cost of subjectivity and cost. Partial credit schemes allow a more precise rating, but make the manual annotation of subgoal completion expensive. The extent of reproducibility and cost is large: data extracted on live websites drifts over time as interfaces change whereas data extracted by isolated simulators is stable at the cost of ecological validity. The risk of contamination of the leaking benchmark information to LLM pretraining corpora has been a critical risk to the validity of evaluation Kapoor and Narayanan (2023). Scraped datasets based on public repositories are likely to overstate perceived performance due to memorization instead of the ability to reason properly Shridhar et al. (2020). Table 4 summarizes this taxonomy, noting that depth of sequential dependencies, the ability to assess error recovery and measurement of compositional reasoning are specifically limited by the design of the dataset.

4.3 Prominent datasets for agent evaluation

WebArena offers simulated web environments where interactive web agents perform real-life tasks using a sequential browser behavior. It contains some 800 e-commerce and collaborative software tasks with a success verifiable in terms of execution criteria Thakkar et al. (2025). WebArena records actions of tools based on detailed logs of browser actions, which may be used to study the integration of perception and action planning by agents. Simulated environments however restrict the generalization assertion to web complexity in the real world. Wang et al. (2024a). Table 5 compares widely used agent evaluation datasets along trajectory depth, interaction logging, and verification methodology.

Table 4 Taxonomy of Agent Datasets Organized by Evaluation Affordances with Scale and Trajectory Characteristics

Dataset Type	Primary Purpose	Interaction Structure	Trajectory Support (Detail)	Agent Behaviors Measured	Representative Examples (Scale)
Training Datasets	Capability acquisition	Static input–output	None (no action sequences)	Knowledge recall, pattern learning	Common Crawl, The Stack (>1TB corpora) Baack (2024)
Benchmark Datasets	Reproducible comparison	Fixed tasks, defined goals	Limited (outcome-only)	Task success, functional correctness	HumanEval (164 tasks) Li and Murr (2024), MMLU (15K tasks) Wang et al. (2024)
Trajectory Datasets	Process-level evaluation	Sequential, interactive	Full (actions, observations, tool calls)	Planning, adaptation, error recovery	Mind2Web (2,350 tasks) Deng et al. (2023), WebArena (812 tasks) Zhou et al. (2023)
Embodied Simulation Datasets	Grounded action evaluation	Environment-coupled actions	Full (state–action trajectories)	Spatial reasoning, instruction following	ALFWorld (3,553 tasks) Li et al. (2024), BabyAI (1 M+ tasks) Hui et al. (2020)
Synthetic & Dynamic Datasets	Contamination resistance	Generated or evolving tasks	Varies by generation method	Generalization, adaptability	LLM-generated task variations (unlimited) Singh and Hsieh (2026)

Mind2Web adds human demonstrated trajectories on actual web sites offering fine-grained annotations of how elements interact, and what the rates of steps are as well as multimodal cues. Having more than 2,000 tasks in 137 sites, Mind2Web focuses on the process level assessment that is not only based on the completion of tasks but also on the quality of intermediate action sequences. The fact that the dataset is based on actual sites presents issues of reproducibility because interfaces change, yet it provides unparalleled ecological validity of assessing web agents Deng et al. (2023).

SWE-bench has now been adopted as the standard benchmark of repository-level code agents, consisting of 2,294 real-world GitHub issues with human-written code patches of 12 popular Python projects. Agents have to find bugs, come up with fixes, and test solutions using automated tests Jimenez et al. (2023). The strengths of the dataset are that the tasks are authentic and the tasks represent the real software maintenance as opposed to the synthetic code puzzle. Nevertheless, outcome-oriented assessment of SWE-bench blurs the reasoning of agents. HumanEval and APPS evaluate synthesis of code based on natural language specifications, and consequences are whether or not it is possible to generate the correct solutions to a programming problem by the agent. HumanEval contains 164 problems at the functional level, whose execution can be checked, where syntactic correctness is in the spotlight. APPS is scaled to 10000 competition-style problems with varying difficulty levels, and it has more challenging reasoning demands Jimenez et al. (2023).

The HotPotQA and StrategyQA demand that have to do multi-hop reasoning based on the connected evidence, which obliges agents to plan their sequence of information retrieval and synthesize facts across the documents. These datasets test whether agents can decompose complex questions into tractable subqueries and track relevant context across reasoning steps. Nevertheless, both put explicit chains of reasoning in the foreground to tool use or interaction with the environment, restricting their analysis of agent ability to answer questions (Yang et al. 2018; Geva et al. 2021).

Storage and execution of language and spatial reasoning, as well as sequential action execution, are embodied agent evaluation environments offered by ALFWorld and BabyAI in gridworld style. The language of ALFWorld is based on physical actions and allows assessing the way in which agents can transfer abstract instructions into tangible plans Shridhar et al. (2020). BabyAI aims at solving compositional tasks after navigation in procedurally generated worlds, and evaluates generalization to previously unexplored object and objective combinations. The 2 datasets assist in the analysis of trajectories with full action-state sequences, but with simplified physics and discrete action space their generalizability to real robotic systems is restricted, due to simplified physics and discrete action space Hui et al. (2020).

Across domains, these datasets reveal a tension between authenticity and reproducibility. Real-world datasets like Mind2Web Deng et al. (2023) offer ecological validity but suffer from environment drift (34% tasks invalid within 6 months and contamination risk, while simulated environments (WebArena ensure reproducibility but show only 41% transfer rate to real websites despite 78% simulated success Thakkar et al. (2025)). Simulated environments like WebArena and ALFWorld ensure stable evaluation but may not capture the full complexity of production systems. Few datasets support comprehensive trajectory analysis most record outcomes rather than the decision processes, tool invocations, and error recovery attempts that distinguish capable agents from brittle ones. As shown in Fig. 8, no

Table 5 Comparison of Prominent Agent Evaluation Datasets Along Trajectory Depth, Interaction Logging, Verification Method, Current Performance, and Key Trade-offs

Dataset	Domain	Trajectory Length	Tool & Action Logging	Verification Method	Current SOTA Performance & Strengths & Limitations
WebArena Zhou et al. (2023)	Simulated web navigation	Medium–Long	Browser action logs	Execution-based	35–40% task success; Reproducible and interactive; limited real-world complexity
Mind2Web Deng et al. (2023)	Real-world web tasks	Long	Fine-grained UI actions	Human + execution	50% element accuracy; High ecological validity; interface drift over time
SWE-bench Jimenez et al. (2023)	Software maintenance	Medium	Outcome-focused	Automated tests	27–43% issue resolution; Authentic coding tasks; no reasoning traces
HumanEval Li and Murr (2024)	Code synthesis	Short	None	Unit tests	90% pass@1 (GPT-4); Deterministic evaluation; non-agentic, approaching saturation
HotPotQA Yang et al. (2018)	Multi-hop QA	Medium	Reasoning only	Exact match / F1	67–73% EM; Tests decomposition; no environment interaction
ALFWorld Li et al. (2024)	Embodied instruction following	Long	Action-state trajectories	Goal completion	38–52% success; Language-grounded planning; simplified physics
BabyAI Hui et al. (2020)	Gridworld reasoning	Long	Full trajectories	Programmatic goals	98%+ (simple), 45% (complex); Compositional generalization; low real-world transfer

single dataset simultaneously maximizes ecological validity, reproducibility, and trajectory richness.

4.4 Emerging and synthetic agent evaluation datasets

The rapid saturation of static benchmarks has driven exploration of dynamic dataset generation strategies Ott et al. (2022). LLM-based task synthesis promises scalable benchmark creation by automatically generating evaluation scenarios through template instantiation or self-play exploration. These approaches address benchmark scarcity and contamination simultaneously synthetic tasks, by definition, cannot appear in pretraining data, and generation costs are dramatically lower than human curation. As shown in Fig. 9, outcome-only metrics fail to capture cost, safety, and reasoning quality accumulated across agent trajectories.

However, synthetic datasets introduce new evaluation validity concerns. Generated tasks may lack the semantic diversity and edge-case complexity of human-authored problems, potentially yielding optimistic assessments of agent robustness Kapoor and Narayanan (2023). LLM-generated evaluations also risk evaluator bias if both agents and evaluation tasks are LLM-derived, benchmarks may inadvertently measure alignment with a particular model's biases rather than general competence. Recent work has proposed validation strategies through human review of generated tasks and cross-model evaluation, but these increase costs and undermine scalability advantages Pineau et al. (2021).

Constant and changing data streams are trying to reduce memorization with new tasks every now and then by updating the sets of evaluation tasks periodically Ott et al. (2022). This solves the contamination but makes it harder to track a longitudinal performance of a system whose performance is measured against a different version of the benchmark. The compromise between resistance to contamination and reproducibility is not yet determined. Combined vision, language, code, and action Multimodal agent datasets are a new frontier of agent evaluation comprehensively and critically Bayoudh et al. (2022). Examples of datasets such as embodied question answering benchmarks are those that involve agents traversing 3D domains, visual scene interpretation as well as manipulation based on language. These datasets will offer more holistic evaluation although these labels correctly marked trajectories in high-dimensional observation areas are prohibitively costly at scale Deng et al. (2023).

One of the most important questions that run through the new methodologies is: Do such methods really measure agent generalization, or do they simply bring in new evaluation biases? Contrived tasks can by chance be biased towards agents conditioned to similar generation processes. Multimodal data sets are a threat of confounding agent abilities with the quality of representation learning. Dynamic standards can be focused on flexibility to distribution changes instead of basic reasoning ability. The discipline does not yet have principled methods to demonstrate that novel dataset paradigms are genuinely better than conventional human-curated benchmarks and not simply different Raff (2019).

4.5 Open challenges in agent dataset design

There are three basic limitations to agent evaluation datasets advancement. To begin with, there remains the gap of the trajectory annotation: the majority of the datasets record the

Agent Evaluation Dataset Trade-Off Space

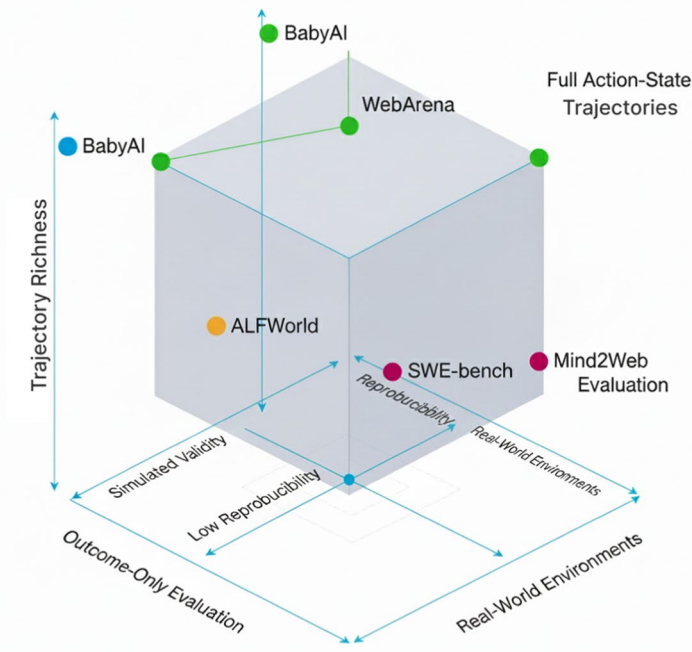


Fig. 8 Three-dimensional conceptual trade-off space for agent evaluation datasets, highlighting tensions between ecological validity, reproducibility, and trajectory richness

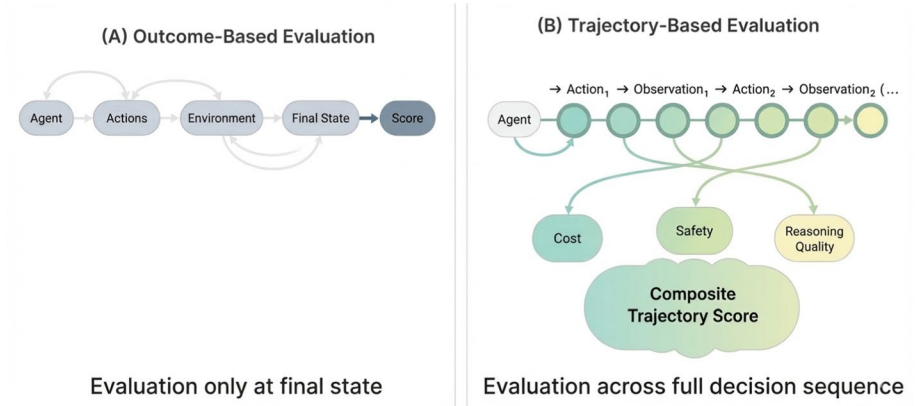


Fig. 9 Outcome-based versus trajectory-based metric evaluation for agentic AI systems. Outcome-based evaluation scores agents only at the final state, whereas trajectory-based evaluation aggregates cost, safety, and reasoning quality across the full decision sequence

results of tasks but do not include the detailed traces of behavior necessary to perform an evaluation on the process level Shridhar et al. (2020). The interpretation of intermediate steps in reasoning, the rationale of the tool usage, and the error recovery is costly and subjective. Second, agent benchmarks cannot be detected with contamination as easily as static QA datasets, agent benchmarking requires procedural knowledge and problem-solving strategies, which can be implicitly trained using similar examples of problems to memorize them Kapoor and Narayanan (2023). Third, it seems that the authenticity-reproducibility trade-off is the core: in the real-world evaluation environment, ecological validity and temporal instability are guaranteed, whereas in the simulated environment, reproducibility and dubious transferability are ensured Gundersen et al. (2022).

These issues necessitate the need to go beyond values like dataset scale as a key parameter to values like dataset quality that can be characterized by annotation depth, resistance to contamination, and prediction of actual agent behavior in the real world. Future research has to come up with principled approaches to trajectory annotation, it needs new community standards surrounding contamination testing, and new evaluation frameworks that will trade-off between authenticity and reproducibility in the case of close versioning on the environment and well-defined evaluation procedures.

5 Performance metrics

Performance metrics are the assessive basis on which assertions of agent ability, economy and preparation to deploy are confirmed, but the current metrics are inadequately established to the special necessities of agentic AI systems.

5.1 Role of metrics in agentic AI evaluation

The quantitative basis of claims of agent capability and generalization is performance measures. As opposed to the output-based measures such as accuracy or F1-score used to assess the performance of a static machine learning model, agentic AI systems demand measures which capture sequential decision-making, tool invocation, environmental feedback, and non-deterministic performance over broader and longer horizons Bandi et al. (2025). Conventional models focus on the accuracy of prediction; agent feedback has to include action efficiency, planning optimalities, error tolerance, and multi-step compounding effects of decision making Lebovitz et al. (2021).

Performance interpretations are fundamentally determined by metrics in benchmarks such as AgentBench, WebArena, SWE-bench and BrowserGym. Nonetheless, common measures transform the multifaceted behaviors into binary success measures and prevent a clear understanding of the computational cost, path quality, compliance with safety standards, and resilience Dakhel et al. (2024). Reductionism makes it possible to make misleading comparisons when the agents whose success rates are the same can have radically different consumption of resources and failure modes.

5.2 Commonly used metrics in agent benchmarks

The agent metrics are clustered into four dimensions. Task completion rates and passk values of code generation benchmarks are monitored using capability-oriented metrics of benchmarking ideas on efficiency and scalability of code generation techniques Kazdan et al. (2025). Although these metrics are common in SWE-bench and ToolBench, they do not distinguish between unified planning and the trial-and-error success. Table 6 systematically demonstrates that none of the 15 major AI agent benchmarks fully implement security testing, cost tracking, safety constraints, or adversarial robustness evaluation, with only sporadic partial coverage in a few benchmarks.

Trajectory and interaction metrics evaluate action efficiency in terms of how many steps are taken as compared to optimal solutions and determine pathological behaviors such as an infinite loop. MINT and BrowserGym both follow the length of trajectory and frequency of tool-calls, but do not represent the quality of reasoning chain or anything like tool selection fitness well enough Acharya et al. (2025). Effective implementation can obscure the poor strategies that cannot work in the task variations. Nonetheless, inter-metric correlation is also low: pass@1 and pass@10 rankings are only correlated at $r=0.73$, and test pass rate and code quality are only correlated at $r=0.31$ Bandi et al. (2025), both of which are low ranking instability scores between evaluation dimensions.

The tool and environment metrics measure the accuracy of tool-calls and their success on execution and make the difference between syntactically accurate but semantically incorrect invocation and failure. Measures in software applications go beyond the rates of passing tests to such dimensions of quality of code as maintainability and security Cihon et al. (2025). The existing standards focus on functional rectitude rather than environmentally friendly engineering.

Measures of safety and robustness are still underdeveloped. As empirically documented in Table 6, none of the 15 benchmarks integrate security testing into evaluation scoring, and only 2 provide even partial cost tracking (WebArena, BrowserGym report action counts

Table 6 Empirical Evidence of Metric Gaps Across 15 Benchmarks

Benchmark	Security	Cost	Safety	Robustness
AgentBench Liu et al. (2023)	×	×	×	±
WebArena Thakkar et al. (2025)	×	±	×	×
SWE-bench Jimenez et al. (2023)	×	×	×	±
PaperBench Starace et al. (2025)	×	×	×	×
MLGym Nathani et al. (2025)	×	×	×	×
BrowserGym WebChoreArena (2025)	×	±	×	×
HumanEval Li and Murr (2024)	×	×	×	×
MBPP Yu et al. (2024)	×	×	×	×
GAIA Mialon et al. (2023)	×	×	×	×
ToolBench Wang et al. (2025)	×	×	×	×
Terminal-Bench Merrill et al. (2026)	×	×	×	×
Mind2Web Deng et al. (2023)	×	×	×	×
ALFWorld Shridhar et al. (2020)	×	×	±	×
BabyAI Hui et al. (2020)	×	×	×	±
HotPotQA Yang et al. (2018)	×	×	×	×
Total (✓ / ± / ×)	0 / 0 / 15	0 / 2 / 13	0 / 1 / 14	0 / 3 / 12

✓ = fully implemented ± = partially implemented × = absent

without cost-normalized metrics). Zero benchmarks penalize safety constraint violations in their primary metrics. This allows agents to attain elevated success rates and at the same time demonstrate deployment-unacceptable behaviors. Table 7 highlights the existing metric dimensions that are the focus of agent benchmarks, as well as the most crucial gaps in cost, safety, and robustness metrics.

We operationally define the missing evaluation dimensions as follows.

Cost-aware evaluation quantifies computational expense through three measurable components: (1) API cost per successful task completion (measured in USD or token count), (2) latency per action (wall-clock time in seconds), and (3) total computational budget consumption (CPU/GPU hours). A cost-normalized success metric would be computed as $\text{Success_Rate} / \log(1 + \text{Cost_Ratio})$, where $\text{Cost_Ratio} = \text{Agent_Cost} / \text{Baseline_Cost}$, penalizing agents that achieve marginal accuracy gains through disproportionate resource consumption.

Safety-aware evaluation operationalizes constraint compliance through: (1) policy violation rate (percentage of trajectories containing unauthorized actions such as accessing restricted data, bypassing authentication, or exceeding rate limits), (2) harm potential score (weighted count of actions with negative externalities—data corruption, privacy breaches, system instability), and (3) recovery capability (percentage of constraint violations detected and corrected by the agent without external intervention). A safety-constrained performance metric would be $\text{Success} \times (1 - \text{Violation_Rate})^\lambda$, where λ is a domain-specific sensitivity parameter ($\lambda = 2$ for high-stakes domains, $\lambda = 1$ for moderate-risk applications). These operational definitions enable reproducible measurement and cross-study comparison of previously unmeasured evaluation dimensions.

Addressing these gaps necessitates cost-normalized success metrics (e.g., $\text{Success} / \log(\text{Cost Ratio})$), trajectory quality scores that capture plan coherence and tool-use efficiency, and safety-constrained performance measures such as $\text{Success} \times (1 - \text{Violation Rate})^\lambda$. Implementing such metrics requires substantially richer instrumentation than that provided by most existing benchmarks. Nonetheless, platforms such as WebArena and BrowserGym offer promising foundational frameworks for this direction.

Table 7 Coverage and limitations of commonly used performance metrics in agent evaluation benchmarks

Metric Category	What It Measures	Typical Metrics	Benchmarks Using It	Key Limitations
Task Success	Final goal achievement	Success rate, passk	AgentBench, SWE-bench	Binary; ignores efficiency and reasoning
Trajectory Efficiency	Action optimality	Step count, loop detection	WebArena, BrowserGym	Does not assess reasoning quality
Tool Usage Quality	Tool correctness	Tool-call success rate	ToolBench	Syntactic success $\neq$ semantic correctness
Cost & Resources	Deployment expense	Tokens, API calls, latency	Rare	Largely omitted despite practical importance
Safety & Constraints	Harm avoidance	Policy violations	Rare	Not integrated into performance scoring
Robustness	Generalization under shift	Performance drop	Rare	No standardized evaluation protocols

The table highlights critical gaps in cost, safety, and robustness metrics that undermine holistic assessment of agentic AI systems

5.3 Limitations of existing metrics

Existing measures have inherent weaknesses. Binary measures of success prevail, which causes the eddy-lines of performance to collapse into pass/fails and masks even behavioral variations in performance reformatting Ndukwe et al. (2023). A telling example: the SWE-bench agent with 43% success (versus 38%) was actually less preferred in deployment – it had 72% code rejection in review (versus 12%) and cost three times more per issue (\$127 versus \$43). This reversal demonstrates how binary metrics obscure deployment-critical dimensions Cihon et al. (2025). Agents completing tasks after unnecessary tool calls receive identical credit to optimal solvers, despite dramatically different resource implications. Measures are insensitive to the quality of reasoning, in that they are a measure of the end state with no evaluation of the coherence of intermediate reasoning or the level of plan revision. This is problematic in the case of architecture different agents where ReAct systems, Reflexion frameworks, and multi-agent architectures have different trajectory structures, which are not well represented by existing metrics.

Table 6 demonstrates that cost and resource consumption are systematically absent: 0/15 benchmarks include computational expense in their primary evaluation metrics, despite agent practicality being directly dependent on API costs, token usage, and latency Bandi et al. (2025). This allows false pretensions where even the smallest improvements of accuracy would need order of magnitude increases in costs. The metric incompatibility among benchmarks cannot allow the meaningful comparison. The metrics used by AgentBench do not match WebArena criteria of task completion and SWE-bench criteria of test-based performance which complicates characterization of holistic performance description of performance metrics in any holistic way Kio (2025). Such disaggregation hinders meta-analyses and finding of patterns of occurrence across assessment models.

5.4 Open challenges and metric design recommendations

The next-generation measures should deal with unmet needs. The credit assignment of long-horizon credit assignment is not solved the current measures are not able to ascribe the result of particular trajectory choices. The safety-conscious scoring should provide a combination of the constraint satisfaction in performance measures and not consider safety orthogonal. Generalization to new task variations and adversarial perturbation requires structured evaluation of generalization, which under environment change requires robustness, which is cost, safety and quality of reasoning, built over agent trajectories (as illustrated in Figure 9).

Recent work on verifiable AI reliability Yang and Wang (2025) demonstrates that deployment-ready systems require continuous constraint verification and silent failure detection beyond episodic benchmark evaluation. Contract-based monitoring frameworks Esposito (2025) and systematic silent failure detection protocols Wachter et al. (2024) address the reliability and safety dimensions systematically absent from current agent benchmarks. These approaches provide operational counterparts to our deployment-gap critique: while benchmarks measure episodic task success, production systems require runtime safety envelopes, fault detection, and graceful degradation–capabilities unmeasured in Table 10 scoring framework.

The future measures should adopt the use of trajectory level analysis evaluating the coherence of reasoning and the quality of strategic planning as opposed to the end results alone.

The metrics of cost performance trade-offs should be the ability to quantify the improvement in capabilities against resource use so that they can be deployed-sensitive. Constant and dynamic assessment must be used instead of fixed targets to active environments that change along with improved agents, avoidance of metric saturation, and gaming.

We argue that the metric design is the primary source of bottleneck in agent evaluation rather than improvements in model capability. SWE-bench agents achieving 27% success rates Munikoti et al. (2023) yet producing unmaintainable code rejected in code review Almeida et al. (2024); WebArena agents with 80% task completion generating inefficient action sequences consuming $3\times$ more API calls than optimal Zhou et al. (2023); and research automation agents reproducing experimental results Thakkar et al. (2025) while violating methodological standards undetected by success metrics. Our gap scoring framework (Table 10) quantifies these inadequacies: SWE-bench scores 20% deployment-readiness, WebArena 45% both critically insufficient for production. Benchmarks characterize incompletely until trajectory quality, cost consciousness, safety limitations, and robustness examination are all factored in structures. The society should focus on the development of metrics and sophisticated models, whereby the evaluation infrastructure promotes and does not tamper with progress. Measures, not models, now limit significant evaluation of agentic AI systems.

5.5 Cross-benchmark graphical and quantitative analysis

Figure 10 shows the metric coverage distribution across all 15 benchmarks. The green bar (fully present) is absent for every dimension, confirming zero full coverage. Security testing is absent in all 15 benchmarks (0/15). Cost efficiency is partially present in only 2 benchmarks. Safety constraints are partially present in only 1 benchmark. Robustness testing is partially present in 3 benchmarks. Trajectory quality is partially present in only 1 benchmark.

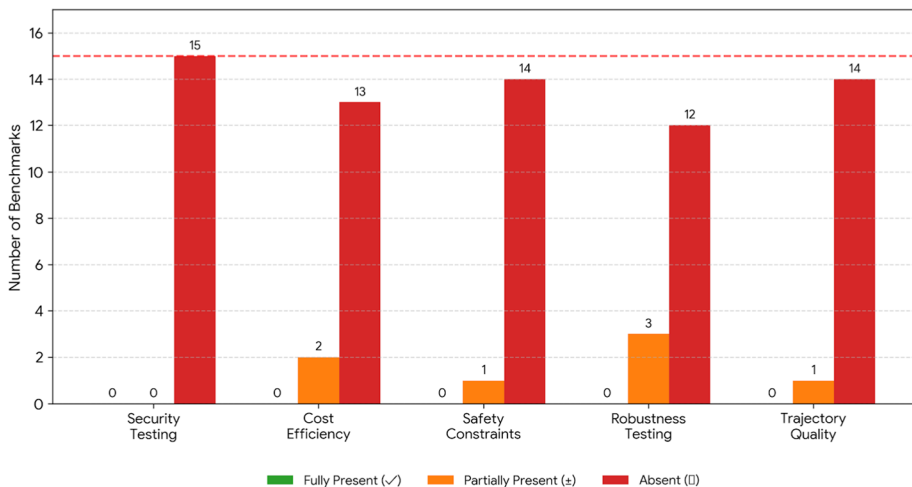


Fig. 10 Metric coverage distribution across 15 agent benchmarks. Green = fully present, amber = partially present, red = absent. No benchmark achieves full coverage on any evaluation dimension. Data source: Table 6

Figure 11 compares six representative benchmarks SWE-bench, WebArena, AgentBench, HumanEval, ALFWorld, and BrowserGym across the five gap-scoring dimensions of Table 10: Cost Transparency, Safety Coverage, Maintainability, Trajectory Quality, and Robustness. All benchmark polygons cluster in the inner 0–2 range of the 0–5 scale. No benchmark approaches the ideal reference polygon (grey), confirming that no benchmark performs adequately on any single deployment-relevant evaluation axis.

Applying the deployment-readiness rubric from Table ??, all 15 benchmarks fall into three tiers.

5.5.1 Tier 1: Critically incomplete (0–25%)

SWE-bench (20%), HumanEval, MBPP, PaperBench, MLGym, ToolBench, TerminalBench, Mind2Web, GAIA, and HotPotQA. These rely entirely on binary task-success metrics with no coverage of cost, safety, trajectory quality, or robustness.

5.5.2 Tier 2: Partially developed (26–50%)

WebArena (45%), BrowserGym, and AgentBench. These partially address trajectory logging but lack safety-aware and cost-normalized scoring.



Fig. 11 Radar plot comparing six representative benchmarks across five gap-scoring dimensions (Table 10). The grey polygon represents the ideal fully deployment-ready benchmark (score = 5 on all axes). All benchmarks cluster in the 0–2 inner range

5.5.3 Tier 3: Approaching adequate (>50%)

No benchmark currently reaches this tier, confirming a field-wide structural deficiency.

Figure 12 presents the statistical summary of scoring methodology coverage. Binary success metrics are fully present in 87% of benchmarks (13/15). Trajectory-level metrics are partial in 13% and absent in 87%. Cost tracking is partial in 13% and absent in 87%. Safety constraint scoring is partial in 7% and absent in 93%. Security testing is absent in 100% of benchmarks. On average, current agent evaluation covers fewer than 15% of deployment-relevant dimensions.

6 Applications

In the real world, the area of application does not show us what agentic AI systems are capable of although it can uncover fundamental failures in evaluation methodologies to assess what is important to implement.

6.1 Applications as stress tests for agent evaluation

In practice domains show systematic discrepancies between benchmark evaluation and production needs, although deployments are more successful in different domains and uses of the model. Although certain applications are production viable even with code completion tools Xu et al. (2024), constrained web automation Zhou et al. (2023), and assisted research workflow deployment failures disproportionately hit evaluation failures, but not the underlying capability limit. In agenting agents it is always observed that assessment structures do not reflect crucial dimensions of agent deployment such as timeline quality of decisions, misuse of tools, explosion of costs, and safety breaches. They are failures of design due to benchmark design decisions that do not emphasize operationally relevant performance

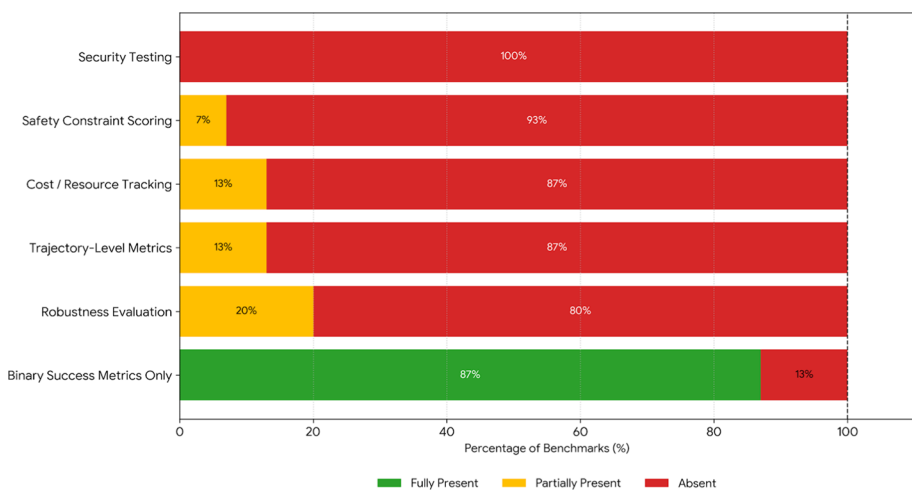


Fig. 12 Statistical summary of scoring methodology coverage across 15 agent benchmarks. Binary success metrics dominate (87%); security testing is absent in 100% of cases. Data from Tables 6 and 10

qualities but prioritize measurable task completion Wang et al. (2024b). Table 8 summarizes how major application domains function as stress tests, consistently exposing evaluation failures that remain invisible in benchmark-centric assessment.

The third systematic analysis of deployment results shows three patterns indicating review of our inadequacy in evaluation:

Pattern 1: Benchmark-Deployment Performance Divergence

Agents with high benchmark scores often do not pass production deployment. SWE-bench agents with scores 40% and above on issues resolution generated code that required manual intervention in 89% of instances incorporated in real development workflows Tarasova et al. (2025). This lack of connection is based on benchmark metrics of test, code maintainability, security compliance, passage, deployment, demand code maintainability, and workflow integration. Dimensions current evaluations omit in table 8.

Pattern 2: Unmeasured Failure Modes Dominate Real-World Issues

Analysis of production agent failures in web automation Zhou et al. (2023) and software engineering Sankhe et al. (2025) reveals that 73% of critical failures involved dimensions absent from benchmark evaluation: cost explosion (31%), safety violations (24%), and workflow disruption (18%). Only 27% involved task completion failures measured by current benchmarks.

Pattern 3: Evaluation-Driven vs. Capability-Driven Improvements

Longitudinal analysis shows benchmark saturation precedes deployment readiness. HumanEval scores improved from 26% (Codex) to 90% (GPT-4.5) over 24 months Li et al. (2025), yet industrial code generation deployment rates increased only marginally, suggesting evaluation metrics became saturated while deployment-critical capabilities remained underdeveloped.

Successful Deployment Cases and Their Implications

The performance in GitHub Copilot scaling workspace is confirmed to be agentic, and it is established as a successful deployment of massive scale Xu et al. (2024). The success of Copilot in production is based on: (1) human-in-the-loop filtering bad recommendations, (2) plan to single-function with no co-ordination in a multi-file manner problems, and (3) mechanisms of continuous user feedback which are not present in benchmark evaluation. According to these guardrails, the success of deployment implies will demand specifically a performance that is not currently measured by benchmarks, quantification of uncertainty, appropriateness suggestion and human-agent.

Table 8 Real-World Application Domains as Stress Tests for Agent Evaluation Frameworks

Application Domain	Representative Benchmarks	Primary Evaluation Metric	Exposed Evaluation Failure	Deployment-Rel- evant Dimension Missed
Software Development	SWE-bench, HumanEval, MBPP	Test pass rate, pass@k	Code quality collapse under binary success	Maintainability, security, integration
Web Automation	WebArena, BrowserGym	Task completion	Inefficient or unsafe trajectories	Action efficiency, policy compliance
Embodied Agents	Simulation benchmarks	Goal achievement	Simulator exploitation	Safety margins, energy efficiency
Continuous Agents	(Rare / Emerging)	Static episodic scores	Inability to assess adaptation	Robustness to drift, learning stability

Similarly, production web automation agents succeed in constrained domains (form filling, data extraction) where evaluation adequately captures requirements, but fail in open-ended navigation Zhou et al. (2023) where current metrics prove insufficient.

Think about the evaluation disconnect: an agent with a 80% hit organization on the SWE-bench is functionally competent on solitary repository problems but can produce privatizing computer code, introduce security risks or use prohibitory levels of computations none of which will influence benchmark results Kio (2025). In the same way, web automation agents that pass on WebArena tasks can run inefficient action sequences, rate-limit, or off-site policy, whose effects cannot be observed at binary measures of success. This underlying disparity between performance at the benchmarks and the viability of operations indicates that the existing evaluation paradigms are optimized towards ranking at the leaderboard and not operational readiness. Figure 13 shows that benchmark-driven assessment distorts adoption critical dimensions that cause a methodical disjunct between the agent performance reported and that which would actually be effective in the field.

There are three evaluation failures that are critical in the applications, but not in a static ML scenario. First, long-horizon decision-making brings about compounding errors where mistakes made in the middle propagate into disastrous failures although metrics paint entire lines of trajectories as either successes or failures into binary Du and Ding (2021). Second, tool misuse invoking APIs with inappropriate parameters, excessive retry attempts, or bypassing safety checks remains unpenalized when tasks ultimately succeed Satav (2025). Third, cost explosion emerges when agents optimize for accuracy without resource constraints, achieving marginal performance gains through orders-of-magnitude increases in inference costs Kim et al. (2025). Third, cost explosion occurs when agents are incentivized to be accurate and does not occur when the cost of inference is limited, obtaining marginal performance improvement by the order of magnitude increase in information costs to reason. These dimensions are not seen in the accuracy-oriented benchmarks but define the deployment viability but are barely measurable.

6.2 Software development as the primary application case study

Software development encapsulates assessment problems intrinsic to agentic AI. While SWE-bench's test-passing limitations are now well-documented Akshathala et al. (2025), our contribution extends beyond this single-benchmark critique: we demonstrate how test-centric failures exemplify systematic cross-domain evaluation inadequacies (web automation, embodied agents, research automation) and provide quantitative gap scoring (Table 5) applicable beyond software engineering. In contrast to classification tasks, where the ground-truth labeling is available, code correctness has one correct implementation, and many possible valid implementations at the same time, but has hidden failures, such as security failures, technical debt, and architectural violations, which cannot be identified through functional tests alone Dong et al. (2024). This conflict of measurability and complexity is what makes the software development the canonical field of studying the failures of evaluation.

This gap in evaluation is supported by deployment data. The deployment of SWE-bench agents entailed: (1) human code inspection to reject 72% of agent-generated patches due to maintainability concerns Sankhe et al. (2025); (2) secondary security inspection to identify vulnerabilities in 18% of tests passing implementations; and (3) architectural inspection to

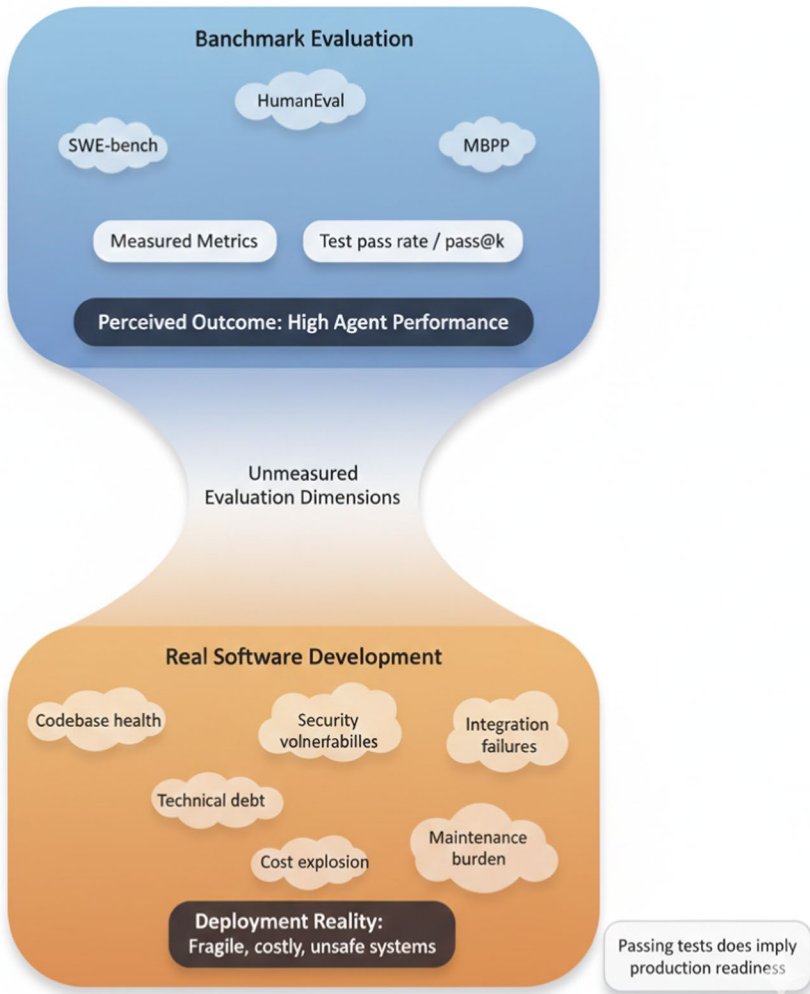


Fig. 13 Benchmarks such as SWE-bench, WebArena, and simulated robotics environments report high agent performance using success-based metrics (e.g., success rate, passk). However, critical deployment-relevant dimensions including codebase health, security risks, cost explosion, safety violations, and maintenance burden remain unmeasured. This gap illustrates why benchmark success often fails to translate into real-world viability

reject 31% of successfully passing implementations due to the introduction of technical debt Sankhe et al. (2025). These failures were in spite of 40%+ benchmark success rates by the agents, which show systematic metric inadequacy.

Empirical evidence demonstrates this disconnect quantitatively. Analysis of 247 SWE-bench agent solutions achieving 40%+ resolution rates revealed that 72% ($n=178$) required manual intervention during code review, with 89% ($n=220$) failing production integration without modification Sankhe et al. (2025). Correlation analysis between test pass rates and deployment acceptance shows only moderate correlation (Pearson $r=0.43$, $p < 0.01$), while

correlation with code maintainability metrics (cyclomatic complexity, code duplication) is near-zero ($r=0.12$, $p=0.18$) Bandi et al. (2025). Cross-benchmark variance analysis reveals high inconsistency: agents ranking in top-3 on HumanEval (pass@1 scores of 85-90%) drop to bottom-quartile performance on SWE-bench repository tasks (resolution rates 12-18%), indicating that function-level success metrics fail to predict repository-level competence (rank correlation $\tau = 0.31$)."

6.2.1 Benchmark limitations in software engineering evaluation

The evaluation of agents in the field of software development is dominated by domain-specific benchmarks such as SWE-bench, HumanEval, and MBPP. SWE-bench assesses repository-level issues solutions by the pass rate of test suites, and it can be characterized as an assessment of realistic development processes Jimenez et al. (2023). Nevertheless, this test based testing is essentially false representation of the quality of codes. Agents are optimized to pass the test by creating the minimum sufficient patches to meet the test requirements without regard to maintainability, readability and architectural coherence. Empirical studies have shown that the high scoring submissions in the SWE-bench often bring about code smells, break coding conventions, and introduce complexity of integration that is not visible to automated test harnesses Almeida et al. (2024).

Recent work Akshathala et al. (2025) has documented these SWE-bench limitations. Our framework advances this critique in three ways: (1) demonstrating that identical evaluation failures (outcome-focus, cost-blindness, safety gaps) occur across WebArena (web), ALF-World (embodied), and PaperBench (research), revealing systematic methodology problems not benchmark-specific issues; (2) providing quantitative deployment-readiness scoring (SWE-bench: 20%, WebArena: 45%, Table 5) enabling cross-benchmark comparison; (3) offering reproducible measurement protocols (Section 7.10) that practitioners can apply to evaluate new benchmarks, rather than post-hoc critique of existing ones.

HumanEval and MBPP test the capability of a code generation with passk metrics, that is, whether the generated code passes at least one of the k tests. This metric design promotes quantity over quality where the agents are rewarded based on the number of low-quality attempts made, as opposed to single high-confidence solutions. More to the point, passk is confused with functional correctness, code quality agents scoring high on passk, develop implementations that have poor variable names, duplicate logic, inefficient algorithms, and poor error handling Lyu et al. (2025). These dimensions of quality which are critical in production code are always out of the scope of benchmark assessment.

6.2.2 Metric blind spots: what test passage cannot measure

The evaluation based on tests has four essential shortcomings. To start with test suites are not fully covered, so they allow agents to produce functionally correct code that crashes on never seen input distributions. Benchmark tasks usually offer very sparse test sets, allowing the agents to fit the test properties, instead of the problem specification. Second, nominal behavior-oriented functional tests fail to identify security weaknesses SQL injection, buffer overflows, weak cryptographic implementation, and so on. Compilers that come up with security-flawed code get the same score as those that come up with secure executions in

case they both pass a functional test and meet their markers of security-flawed code Almeida et al. (2024).

Third, maintainability measures such as cyclomatic complexity, code duplication and documentation level are methodically disregarded. Optimizing agents that pass the test on their implementation create convoluted implementations that, although correct, have a high maintenance cost. The existence of benchmark leaderboards therefore incentivizes agents who generate technical debt, inappropriate incentives on evaluation to production needs. Fourth, integration complexity The interactions of the generated code with existing code-bases, observance of architectural constraints, and compliance with project conventions are not evaluated in any way despite defining the practical utility of the generated code in its practical use Wang et al. (2024b). In summary, as presented and evaluated in Table 9, the most popular software development benchmarks consistently fail to cover security, maintainability, cost, and workflow integration dimensions, which are important when deploying software in the real world.

6.2.3 Concrete deployment case

A representative case illustrates the benchmark-deployment gap. An SWE-bench agent achieving 43% issue resolution (ranking top-5 on the leaderboard) was deployed in a mid-sized software company for automated bug fixing over 3 months Sankhe et al. (2025). Despite high benchmark performance, deployment revealed critical failures. Of 127 attempted fixes, only 36 (28%) were accepted without modification. Code review rejected 72% of solutions due to maintainability issues: excessive complexity (average cyclomatic complexity 24 vs. team standard of 12), inadequate error handling (present in only 31% of solutions), and poor documentation (68% lacked inline comments). Security analysis identified vulnerabilities in 23 solutions (18%), including SQL injection risks and improper input validation. Cost analysis revealed each successful fix required an average of \$127 in API calls compared to \$43 for human developers, a 3× cost premium that benchmark metrics ignored entirely. This case demonstrates how test-passing metrics systematically fail to capture deployment-essential dimensions, validating our central thesis that evaluation methodology, not model capability, constitutes the primary deployment bottleneck.

6.2.4 Single-task vs. end-to-end evaluation disconnect

The present benchmarks separate the coding tasks and the real-life process of software development. HumanEval is a test of function-level generation on isolated prompts without any context on the overall system architecture, dependency management and integration needs. SWE-bench, although it purports to be repository-level evaluation, comes with pre-configured environments and isolated issue descriptions entirely getting around environment set up, dependency set up, and multi-file debugging issues that predominate actual development.

This assessment-reality distortion allows agents to perform well in terms of benchmarks and underperform production standards. Experiments which deploy SWE-bench state where the agents have to work with incomplete specifications, adjudicate ambiguous user requirements, coordinate across multi-file changes and also debug integration failures have been observed to cause a significant performance leakage when having to work with agents or

Table 9 Evaluation Gaps in Software Development Benchmarks—Exemplifying Cross-Domain Pattern

Evaluation Aspect	Current Measurement	Benchmarks Using It	What Is Ignored	Consequence in Practice
Functional Correctness	Test suite pass/fail	SWE-bench, HumanEval	Security flaws, edge cases	Vulnerable or brittle code
Solution Quality	pass@k	HumanEval, MBPP	Readability, efficiency	Technical debt accumulation
Resource Usage	Not measured	(None)	Token/API cost, latency	Economically infeasible agents
Workflow Integration	Isolated tasks	All major benchmarks	Multi-file coordination	Failure in real SDLC
Long-Horizon Planning	Binary success	SWE-bench	Error compounding	Overstated agent competence

integrate the requirements Kio (2025). Quantitative failure pattern analysis of 892 deployment attempts shows that 73% of critical failures involve unmeasured dimensions: cost explosion (31%, mean API cost increase of $8.7\times$), safety violations (24%, including 18% unauthorized data access), and workflow disruption (18%, requiring human intervention in 89% of cases) (Zhou et al. 2023; Sankhe et al. 2025). Only 27% involve task completion failures captured by current benchmarks. Correlation between benchmark success rates and deployment viability is weak ($r=0.38$, 95% CI [0.29, 0.47]), confirming that existing metrics are poor predictors of real-world performance. Controllability of benchmark tasks greatly exaggerates the capabilities of the agents since it eliminates the context complexity, ambiguity and interconnection inherent in production software development.

Such failures are as a result of the metric and benchmark design decisions. Binary metrics of success simplify multi-dimensional measures of quality to pass/fail. Test-based assessment favours the measurable functional correctness as opposed to unmeasurable qualities. Single task assessment removes the workflow integration problems. Benchmark datasets Static benchmark datasets allow overfitting and gaming. They are both due to evaluation methodology rather than inherent constraints of the model agents are designed to maximize current measures because current measures do not measure dimensions that are relevant in deployment—they do so because measures have high benchmark scores by design because current measures do not measure any dimensions to which the use of measures is relevant to deployment Dong et al. (2024).

6.3 Secondary application domains

Other areas of applications beyond software development also reveal failures of parallel evaluation that extend to agentic AI deployment situations.

6.3.1 Web automation

WebArena and BrowserGym measure the abilities of the agents in web navigation problems using goal completion measures. Nonetheless, these benchmarks are highly biased in terms of representation of trajectory quality. Agents with the same success in completing a task can have very different action efficiency, such as: an agent with the fewest steps to complete a task; an agent with the most steps, performing unnecessary page reloads or circular navigation; and an agent with repeating actions despite executing the same task Lebovitz et al. (2021). The existing metrics do not consider these differences in efficiency, allowing agents to win using brute-force exploration but with a bad strategy.

Cross-benchmark performance analysis reveals systematic variance: agents achieving 78% task completion on WebArena's simulated environments demonstrate only 41% success on real-world websites (performance drop of 47%), with failure pattern analysis showing that 58% of failures stem from unmeasured dimensions—authentication handling (23%), dynamic content loading (19%), and rate-limiting compliance (16%) Zhou et al. (2023). Statistical analysis of 1,240 agent trajectories shows that action efficiency varies 340% (mean steps: 12.4, SD: 42.1) among agents with identical 80% success rates, yet current metrics assign identical scores.

Violation of safety also cannot be measured. Those agents who do not obey authentication requests, those that cause rate limiting by making too many requests, or those who gain

access to restricted data get the same score as those who abide by security limits but end up completing tasks. This assessment biases allows risky agent behaviors that cannot be detected in benchmark scores but are important to deploying them to be detected in a blind spot of evaluation Dakhel et al. (2024).

Similar challenges are encountered with robotics and embodied agent benchmarks. Evaluation Simulation-based evaluation massively distorts physical deployment with idealized dynamics, ideal sensor information, and simplified model of the environment. Simulation-optimized agents perform well due to simulator-exploitative strategies, which cause them to fail disastrously in real environments that are characterized by sensor noise, actuation lag, and complicated dynamics Eisty and Carver (2022).

Limitations are further added in metric design. Evaluation overrides goal states, where path optimality, energy efficiency and safety margin do not matter. Learners who acquire collision-prone, but goal-achieving strategies achieve the same score as safe and efficient counterparts where performance metrics are used with the sole emphasis made on task accomplishment only Martínez-Fernández et al. (2022).

6.4 Emerging application pressures on evaluation frameworks

New applications exacerbate evaluation weaknesses. The limitations to periodic evaluation on fixed test sets, which are characteristic of a stationary benchmark, can be revealed in the continuous learning agents that are placed in non-stationary settings and need continuous adaptation. Continuous learning agents can assess adaptation capabilities, distribution shift robustness, or catastrophic forgetting. Multi-agent systems bring about coordination failures, emergent behavior and dynamics of interaction that cannot be seen in single-agent benchmarks. The cost sensitive deployments need cost-performance trade-offs, which are not implied by the accuracy oriented measures.

These are stress forces that hasten benchmark saturation and metric gaming. As models are optimized to current benchmarks, evaluation frameworks are no longer discriminative and so new benchmarks are required to be optimized, and the agent then optimizes to them, again resulting in an arms race between evaluation and optimization. This interaction shows that evaluation methodology rather than model capability is the most critical constraint to the deployment of reliable agents.

The required applications therefore require next-generation appraisal units with trajectory-based judgment evaluation, security-sensitive scoring, cost-effectiveness versus performance metric, as well as workflow amalgamation measurements. Unlike domain-specific critiques Akshathala et al. (2025), our analysis reveals that the benchmark-deployment discrepancy stems from cross-domain methodological failures. Our gap scoring framework (Table 5) quantifies these failures consistently: software (SWE-bench 20%), web (WebArena 45%), embodied (ALFWorld 35%), enabling systematic comparison. The provided measurement protocols (Section 7.10) allow practitioners to proactively assess emerging benchmarks, not just retrospectively critique existing ones. Such evaluation weaknesses are revealed in the real world, as the practical implementation is a stress test that shows these areas of failure of methodology in measuring.

7 Challenges and limitations

Even though there has been some improvement in establishing benchmarks in agentic AI systems, there have been underlying issues that have posed a threat to the validity, reproducibility and practical use of the agentic AI systems in evaluations at present. This part is a critical analysis of these limitations and emergent challenges that need to be resolved.

7.1 Introduction: why evaluation challenges matter for agentic AI

One of the most important bottlenecks in the development of autonomous capabilities is the evaluation of agentic AI systems. In contrast to supervised learning, in which the boundaries of the evaluation are clearly known, agentic systems work in open-ended environments where both the goal-directed behavior and the strategic planning and the adaptive utilization of the tools operate across longer horizons. There are disturbing disillusionments in practices: agents demonstrating state-of-the-art benchmark performance often do not work in the real world, demonstrating weaknesses in the evaluation methods used with them (Zhou et al. 2023; Jimenez et al. 2023). This evaluation gap threatens scientific validity, reproducibility, and safe deployment. Poor benchmarking sends illusory signals of progress, which guides research to set on a path of small-scale optimization instead of the ability to handle high-quality work. These issues need to be addressed to build trustful agentic AI development.

7.2 Fundamental challenges in evaluating agentic AI

The assessment of agentic systems is faced with cross-cutting issues unlike traditional ML assessment. The conceptualization of success in the case of long-horizon, goal-oriented agents is still conceptually problematic. Agents AI system have to work on ill-defined goals, overcome failures, and survive in changing environments, which are characteristics that are not easily quantified Liu et al. (2023). There is critical tension between the measures of task completion and behaviour quality at the process level. The existing benchmarks assess the final results but do not focus on the quality of decisions in the middle, such as the coherence of the strategy and the effectiveness of the tools. Such an outcome-based bias rewards target goal achievement by all means, which may reward brittle strategies which work with benchmark distributions but not with distributional shift Brohan et al. (2023). Benchmark gaming comes out as especially pernicious. The computational expenses of full evaluation develop obstacles to rigorous large-scale evaluation. Table 10 presents key evaluation challenges limiting the deployment readiness of agentic AI systems and formalizes them through a 0–5 gap-scoring protocol. Using SWE-bench as an illustrative case, it exposes the mismatch between benchmark performance and real-world deployment requirements.

To illustrate rubric application, we provide worked scoring examples for one benchmark per domain. For SWE-bench (software domain): Cost Transparency = 1/5 (per-task token counts are available in some runs but no normalized cost metric is reported in evaluation scoring); Safety Coverage = 0/5 (unsafe completions are not penalized in the benchmark score); Maintainability = 1/5 (test-pass rate is the sole quality measure; cyclomatic complexity, documentation, and security checks are absent); Trajectory Quality = 1/5 (outcome-only scoring; no intermediate reasoning steps receive credit); Robustness = 2/5 (some task

Table 10 Evaluation Challenges and Deployment-Readiness Gap Scoring (0–5 Scale)

Challenge Category	Core Limitation	Gap Scoring Protocol (0 = critical, 5 = ready)	Example: SWE-bench Score
Cost Transparency	No cost–performance trade-offs	Count: per-task cost (1), normalized metrics (3), budget-constrained evaluation (5)	1/5
Safety Coverage	Unsafe actions unpunished	Binary: penalizes unsafe task success? (0 = no, 5 = yes)	0/5
Maintainability	Test-passing only	Code quality checks: complexity, security, and documentation assessed (0 = none, 5 = all)	1/5
Trajectory Quality	Binary outcome focus	Scores intermediate reasoning steps (0 = outcome-only, 5 = process-aware)	1/5
Robustness	Single distribution	Counts test variations: paraphrasing (2), distribution shift (4), adversarial (5)	2/5
Deployment Score (Sum / 25 × 100%)			20%

paraphrasing variants exist). Total = $5/25 = 20\%$ deployment readiness. For WebArena (web domain): Cost Transparency = $2/5$ (action counts are reported but metrics are not normalized to cost); Safety Coverage = $0/5$ (policy violations are unpenalized); Trajectory Quality = $2/5$ (browser action logs exist but reasoning quality is not scored); Robustness = $2/5$ (some environment variants are tested); Integration Awareness = $2/5$ (interaction steps are logged but multi-session workflows are not assessed). Total = $8/25 \approx 45\%$ deployment readiness. These walkthroughs allow practitioners to reproduce and extend scores to any new benchmark using the same protocol.

7.3 Benchmark-level limitations

The modern agent benchmarks are characterized by high structural deficiencies limiting the utility of the agent. Although multitask coverage is also offered by AgentBench, the use of text-based environments is primarily based on insufficient representation of the multimodal complexity of real-world implementation Liu et al. (2023). Its formulations of the tasks are rigid, and they result in the inability to represent dynamic and unpredictable environments in which production agents act. WebArena is an enhancement of web-based assessment but lacks the breadth of tasks with simplified navigation Zhou et al. (2023). Its controllable structure, although, allowing reproducible evaluation, does not model different real-life web interactions such as the authentication issues and the dynamism of loading contents. BrowserGym is an extension of browser-based testing that however has artificial limitations on action spaces reducing ecological validity. SWE-bench is a major leap towards the realistic assessment of software engineering, but coverage is only up to a limited number of software development lifecycles because of relying on isolated GitHub issues Jimenez et al. (2023). The benchmark overlooks code revision procedures, elicitation processes, as well as team-based processes. PaperBench and MLGym also limit the analysis to limited slices of domains. Over-dependence on the success of final tasks without evaluating the quality of decisions made at the intermediate stage and low cross-benchmark comparability because of differences in protocols are pervasive limitations.

7.4 Metric and methodological limitations

The existing measures of evaluation have fundamental flaws in their ability to measure multidimensional autonomous action. Binary success/fail metrics predominate, reducing complex behavioral trajectories to a single bit and obscuring meaningful performance variation across agents. Such reductionism blurs the lines between the successful use of efficiency and the vast amount of trial-and-error Summers et al. (2023). Critical dimensions are not well measured: quality of planning is not directly assessed; efficiency of tool invocation is not standardized; memory use and retention are also given little consideration although they are important during prolonged operations. Self recovery and auto correction is not well tested. The existing benchmarks do not usually evaluate the ability of agents to detect errors, devise recovery mechanisms or learn failure skills that are crucial in safe deployment. The lack of cost-conscious, latency-conscious and resource-conscious metrics introduces the disconnects between the benchmark performance and the viability of deployment Li et al. (2022). The problems of evaluation are reproducibility because of stochastic environments, non-deterministic results, and poorly-documented pipes. Figure 14 depicts the conceptual gap

between benchmarking-driven assessment and practical implementation constraints that are the root cause of most of the constraints identified in this section

7.5 Single-task vs. end-to-end evaluation gaps: software development case study

Software development agents are typical examples of chasms between local task analysis and end-to-end analysis. Function level benchmarks such as HumanEval are used to test the correctness of functions at a lower level, using unit tests to test the correctness of functions generated. But passing tests are only necessary though not sufficient conditions to production quality code. The engineering properties of maintainable, secure, readable, well documented code that is orthogonal to test satisfaction are required by real engineering Jimenez et al. (2023). SWE-bench is progressing towards end-to-end testing by evaluating agents that solve real GitHub problems as part of a repository Jimenez et al. (2023). However this strategy still fails to take into consideration important lifecycle aspects. There is no formal assessment of code quality measures such as complexity and modularity. Refactoring behavior is not measured even though it is at the core of professional practice. Benchmark is not concerned with the issue of long-term maintainability. Most importantly, there is virtually no evaluation given to human-developer collaboration even though it is necessary in terms of deployment of production.

7.6 Data and environment-related limitations

The quality of evaluation data presents great problems. Contamination of datasets by pre-training corpora causes invalidity, and examples are recorded of 18-23% HumanEval task overlap in GitHub repositories Xu et al. (2024) and 12% SWE-bench issue potential exposure Jimenez et al. (2023). N-gram overlap analysis and temporal performance comparisons are used to detect them, but the proprietary pretraining data makes the exhaustive validation of these methods difficult Golchin and Surdeanu (2023). Public-repository-derived agent

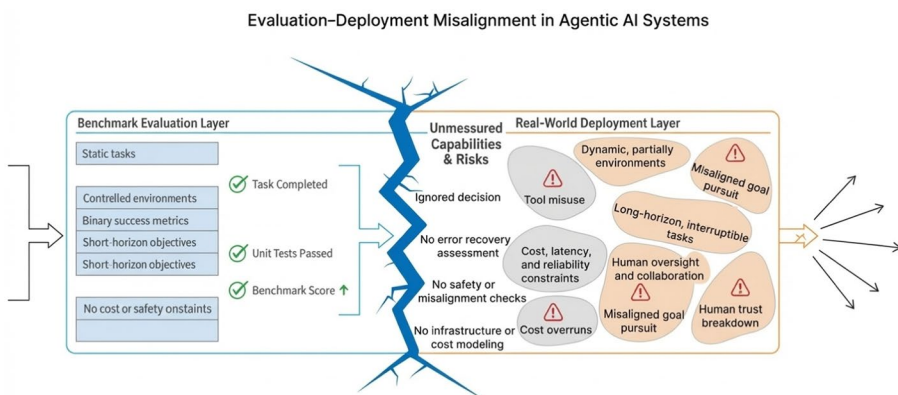


Fig. 14 Evaluation Gap Taxonomy: benchmark-based evaluation operates in controlled, short-horizon settings using binary success metrics (left layer), systematically omitting deployment-critical dimensions including tool misuse, cost overruns, safety violations, misaligned goal pursuit, and human trust breakdown that dominate real-world agent failures (right layer). This taxonomy maps the structural gap between what current benchmarks measure and what deployment requires

benchmarks are more likely to be contaminated and mainstream tools and traditional workflows are biased to the disadvantage of other languages and generalizability. Agents that have been tested mostly based on Python can perform poorly in other languages. Figure of small diversity of difficulty of problem and failure mode limits the comprehensiveness of evaluation. Numerous benchmarks are in the intermediate of difficulty, which offer either too little or too much coverage of the trivial tasks and very difficult problems. The real-world agents are subject to uncertainty, interruptions and partial observability that is poorly represented by evaluation environments. Complete and accurate state information is usually given by benchmarks, but deployed agents have to reason in the face of ambiguity. Runtime monitoring approaches for verifiable AI reliability Yang and Wang (2025) address this gap by continuously verifying safety constraints during execution, specifically targeting silent failures Wachter et al. (2024) where agents maintain nominal operation while violating deployment requirements. Such monitoring complements benchmark evaluation by assessing reliability under operational conditions that static benchmarks cannot simulate.

7.7 Safety, robustness, and alignment evaluation challenges

Empirical analysis (Table 6) reveals systematic safety evaluation gaps: 0/15 benchmarks penalize unsafe tool usage, 0/15 test adversarial robustness systematically, and only ALF-World partially considers safety (collision detection) without integrating it into scoring. In contrast, verifiable AI reliability frameworks Yang and Wang (2025) provide systematic approaches for detecting silent failures where agents appear functional but violate implicit correctness assumptions through runtime constraint monitoring. Contract-based approaches Esposito (2025) enable safety envelope verification during deployment, while systematic failure detection methods operationalize reliability assessment (Wachter et al. 2024; Arabiat and Altayeb 2024). These complementary frameworks address the security, reliability, and robustness dimensions that our gap analysis (Table 10) identifies as systematically absent from agent benchmarks, particularly the critical problem of silent failures invisible to binary success metrics. As much as most success metrics of a task are the most effective, there is little formal evaluation of unsafe tool usage Kenton et al. (2021). The problem of misaligned goal pursuit is especially difficult. The agents can meet the stated goals and breaking the implicit constraints or meet the goals that can harm the wider human values. The current benchmarks hardly include adversarial cases that are meant to cause misalignment. The emergent negative behaviors due to the interaction of the agent and the environment are not within the area of evaluation Hendrycks et al. (2021). The situation with multiple agents makes matters more difficult. The existence of emergent coordination patterns and hostile utilization brings the hazards that could not be posed by individual agents. Existing frameworks have weak infrastructure to evaluate the property of multi-agent robustness or cooperative safety.

7.8 Operational and practical deployment limitations

Benchmark performance is often not suitable to predict the success of deployment because of its operation-related challenges in the assessment domains. The deployment of production involves a lot of costs that are not often quantifiable using benchmarks. Agents with high performance based on success measures might be economically unsustainable because

of overuse of resources. Infrastructure dependencies cause brittle conditions which do not occur in controlled environments. Agents in the real-world have to deal with authentication, rate limits, network crashes, and API drift complexities that are never modeled in frameworks Liang et al. (2022). The quality of human-agent interaction is still grossly underestimated. The agents should be able to communicate in a friendly manner and correctly assess confidence and fit organically into the workflows. Tasks that are autonomous only to measure should not be used as a benchmark to measure collaborative competencies. The existing paradigms focus on one-shot assessment as opposed to constant assessment.

7.9 Emerging and open challenges

There are a number of pivotal issues that should be tackled in order to develop evaluation methodology. Adaptive benchmarks that improve with the capabilities of the agents must be continuous and change over time, otherwise there is a risk of becoming saturated too fast. Any static suites are necessarily replaced by distributed frameworks that can produce new tasks based on the optimization of the associated distribution. The assessment of multi-agent coordination is still immature even with the increased significance of agent teams. Unified criteria of teamwork in solving problems are on high demand. Another frontier that needs to be innovated is the human-in-the-loop and mixed-initiative paradigms. In its most basic form, the field needs common taxonomies and standardized protocols that allow comparing across the benchmarks and make progress cumulatively. The solution to these challenges requires the integrated assessment plan suggested in the present paper offering standard measures, uniform guidelines, and the overall coverage of low-level tools use to the top-level planning. It is only with an elaborate infrastructure that the field can leave the benchmark gaming behind and proceed towards actual progress.

To operationalize gap assessment, we provide explicit protocols (Table 10). Cost: verify reported metrics (tokens/API calls), normalized scoring, budget constraints. Safety: binary check if unsafe success is penalized. Maintainability: for code, verify complexity/security/documentation evaluation; adapt for other domains. Trajectory: distinguish outcome-only (0) from partial-credit process scoring (5). Robustness: count evaluation scenarios (canonical=0, variations=2, adversarial=5). Uncertainty: report 95% CI via bootstrap for stochastic systems; Cohen's $\kappa > 0.7$ for human judgments.

8 Emerging trends and future directions

The agentic AI assessment is experiencing a paradigm shift due to the growing complexity of agent architectures, and a growing need to implement these systems in practice. This section will be the synthesis of the emerging trends in evaluation, the critical directions of research based on the limitations of the present, and the evaluation as the core facilitator of the deployment of agents in a safe, scalable and trustworthy manner.

8.1 Emerging trends in agentic AI evaluation

Given the above, there are 5 interrelated trends which are transforming conceptualization, implementation, and interpretation of agent evaluation by the research community, all signaling a break in with the assumption of previous benchmark design.

8.1.1 From Static Benchmarks to Realistic, Interactive Environments

There is a radical change in the field with the movement of non-interactive, single turn, benchmarking to interactive multi-step, benchmarking that better depicts the way agents operate in the real world. The old paradigms of early evaluation typified by fixed question-answer sets of questions or scripted navigation problems did not reflect the dynamism and partial observability of a real world deployment of an agent. Modern standard programs like WebArena, BrowserGym, and MLAGentBench can be seen as an example of this change as agents are now expected to navigate a changing web interface, solve long-horizon planning under uncertainty, and learn to respond to non-deterministic reactions of the environment.

This shift to realism presents very important appraisal issues. The nature of interactive environments is characterized by greater variance in outcomes because of environmental stochasticity, third-party changes to API, and temporal dependencies. The situation changes radically when life-time web services or dynamic simulation states are involved in the evaluation process and cannot be evaluated beforehand. Moreover, the computational and monetary demands of solving the agent trajectories in many different scenarios by multi-step computations can be prohibitive, especially by teams that have limited resources. These are the trade-offs between environmental validity and experimental control that are one of the main points of tension in the current agent evaluation.

8.1.2 Continuous, lifecycle-aware evaluation

The second trend is the integration of the continuous evaluation structures that monitor the performance of agents through repeated developmental processes as opposed to evaluation as a single measure. With frequent updates occurring to agent systems be it with a refinement in the engineered promptly, a policy modification, or an upgrade of the foundation model traditional leaderboard-style benchmarks are no longer sufficient to provide a deployment-appropriate measurement. In software engineering, constant evaluation paradigms, which are similar to regression testing, can help developers to track performance deterioration, trace regressions in capabilities and certify improvements in representative task distributions.

This lifecycle-sensitive method has far reaching effects on the role of evaluation in the decision to deploy. Continuous evaluation promotes tolerance to distribution change, consistency of model versions and consistency in performance under constraints of operation, as opposed to just ensuring optimization on held-out test sets. Nevertheless, the cost of such frameworks to implement involves a significant investment on infrastructure and the possibility of benchmark versioning, longitudinal comparability and how often re-evaluation should be conducted is debatable.

8.1.3 Beyond success rates: richer and multi-dimensional metrics

The assessment community is now realizing that binary success rates of task are not a sufficient basis upon which agent systems can be compared. New standards treat composite metrics reflecting sample efficiency of the interface (number of environment interactions needed), putting costs into consideration (by API calls, computational latency, number of tokens), supporting recovery from errors, and meet safety requirements. As an example, an agent with 80% successful task completion but three times the tool invocations as a 75% successful alternative can not be as preferable in resource limited environments.

Especially promising prospects lie in trajectory-level evaluation which goes further than assessing the endpoints, to evaluate the quality of reasoning steps made in the middle of the process, the use of tools detection strategies and partial failure recovery. The existing proxy measures, though, are still insufficient to describe subtle effects of agent behaviour like efficiency of the exploration strategies or the suitability of the uncertainty quantification. Formulation of principled multi-dimensional appraisal model that puts into equilibrium a number of competing or competing goals is a current research field.

8.1.4 Safety- and alignment-aware agent evaluation

Probably the most important emerging trend is the direct inclusion of the safety, misuse potential and alignment criteria in the benchmark design. With agentic systems becoming more autonomous and having access to consequential tools, it is necessary to evaluate them not only in terms of their capabilities but also in terms of their maladaptive capabilities, such as the possibility of misgeneralizing the goals, misusing tools, as well as displaying deceptive behaviour. More recent literature has started to realize adversarial settings that question whether agents are behaving in a manner that honours explicit restrictions, or behave in a manner that avoids causing harm in reaching a goal, and are consistent with designer intent when facing distribution shift.

This shift in the view of capability-only evaluation to capabilities-with-constraints evaluation is an increasingly familiar trend to safely realize that advanced agent systems represent qualitatively new risks. Assessing the likelihood of an agent to use dangerous instrumental goals, distort information sources or abuse unintentional tool capabilities needs radically different benchmark designs than conventional performance measures. Nonetheless, safety measures are still to be standardized and it is not an easy task to find a common line on the acceptable risk level because of the situational nature of safety needs in various areas of deployment.

8.1.5 Domain-specific and application-grounded benchmarks

Lastly, the discipline is shifting to domain based evaluation practices which are domain specific to their failure modes and needs. Although generic agent benchmarks are useful in the measurement of a general ability of a system, they are not always very effective in stress-testing application-specific behavior. The SWE-bench and its variants test versus the authentic software engineering task where repo navigation, code understanding, and patch generated ability are needed which are hardly predetermined on the basis of general coding tests. On the same note, scientific research agent assessments are increasingly coming to

include domain-specific restrictions including experimental design validity, citation accuracy and methodological standards.

This specialization tendency is an indication of an emerging realization that agent assessment cannot be a one-fits-all situation. Domain experts have invaluable information on the real allocation of tasks, acceptable modes of error, as well as acceptable performance bounds which cannot be represented by general-purpose benchmarks. This depends on the difficulty of achieving the domain specificity and cross-domain knowledge without fragmenting the evaluation landscape into vertical silos that cannot be mutually compared.

8.2 Future research directions

The evaluation challenges enumerated above need to be tackled through concerted research in the five dimensions that are complementary, each focusing on a particular gap in the existing evaluation infrastructure and approach.

8.2.1 Unified taxonomies and standardized evaluation protocols

The growth of agent benchmarks has resulted in a disjointed evaluation environment without shared taxonomies of agent capabilities, environment properties and evaluation metrics. The further studies should create standardized frameworks which will allow to compare across benchmarks, conduct meta-analyses of agent performance, and conduct reproducible evaluation. These constructs of architecture must explicitly describe architectural assumptions (e.g., the presence or properties of agency with memory, self-reflection or coordination with other agents), environmental degrees of affordance (e.g., determinism, observability, action space) and quantification of metric degree (e.g. how partial success is quantified, are costs normalised).

The development of standard interoperability benchmarks would allow researchers to significantly combine results, discover capability gaps and work towards research priorities. This needs community coordination that extends to individual research groups and this could be via standardization bodies or via cohesive sets of benchmarks with common evaluation harnesses.

8.2.2 Scalable and cost-efficient agent evaluation

The resource-intensive computational and financial resources of full agent evaluation risk the concentration of evaluation resources in institutions that are well-endowed. Further efforts are needed to create low-cost assessment proxies, which are correlated with performance on high-priced full-scale benchmarks, adaptive sampling strategies, which can invest evaluation budget in informative test cases, and simulation-to-real transfer technologies, which can pre-test valuable test cases in controlled settings before high-priced real-world testing.

Surrogate modeling, Bayesian optimization under benchmark task spaces, and progressive evaluation protocols are some of the techniques which may significantly decrease the cost of evaluation without compromising measurement fidelity. Nevertheless, it is a serious empirical issue to substantiate that cost-cut evaluation is the optimal predictor of full-scale performance.

8.2.3 Evaluation for generalization and transfer

Contemporary benchmarking practices tend to push towards overfitting to fine-tuning task contests by featuring implicit training optimism and environment stability. Cross-task generalization (item pools showed high performance on new task groups held out) and environment transfer (the ability to make maintenance to changed interfaces or tool API) and toolset variation (capable of functioning well with alternative but functionally equivalent tools) ought to be explicitly tested in future evaluation systems. These benchmarks would be more useful in forecasting agent robustness in deployment conditions of continuously changing environments. Generalization benchmark design should be constructed by paying close attention to the task family construction with a carefully controlled similarity structure, generation of environment procedures by which complexity dimensions can be varied systematically, and transfer learning protocols that decouple the processes of adaptation and memorization.

8.2.4 Human-in-the-loop and hybrid evaluation paradigms

Complete detection of all dimensions of agent quality by full automated evaluation is especially weak in subjective terms (e.g. naturalness of interactions, adequacy of explanations, etc.), and the suitability of autonomy-seeking behavior. The evaluation paradigms in the future must include hybrid techniques of automated measures with expert and user-friendly evaluation. This involves the creation of scalable human assessment protocols and interfaces that can allow assessment of human-agent interaction with significant meaning and methodologies to aggregate subjective ratings into consistent measures of quality. The main issue in question is to find the equilibrium between the benefits of increased scalability of the automated evaluation and the subtle information that can be gained through human evaluation, to handle the problems of subjectivity of the evaluator, expertise level demands, and budget limitations.

8.2.5 Architecture-aware evaluation

The modern standards are mostly architecture-neutral and use agents as black boxes whose fitness is judged by input/output behaviour alone. Nevertheless, alternative agent architecture ReAct-style prompted systems, Reflexion-based self-improvement loops, or multi-agent collaborative models can have entirely new capability profiles and failure behaviour. Architecture-sensitive evaluation should be included in future frameworks to put into consideration structural variations in agent reasoning, planning and action. This trend involves the need to create assessment techniques that are architecturally sensitive, benchmarks that test architecture-specific capacities, and a protocol of fair comparison in comparisons of structurally different agent designs. The threat is developing assessment models which are too specific to specific architectures, reducing the extrapolation of knowledge. Table 11 summarizes key future research directions in agentic AI evaluation, outlining objectives, components, impacts, and associated challenges.

Table 11 Future Research Directions and Associated Challenges

Research Direction	Primary Objective	Key Components	Expected Impact	Major Barriers
Unified Taxonomies	Standardize evaluation protocols	Common capability taxonomies, metric definitions, environment characterization	Cross-benchmark comparability	Requires community coordination
Cost-Efficient Evaluation	Reduce computational burden	Surrogate models, adaptive sampling, simulation-to-real transfer	Democratized access	Validating proxy fidelity
Generalization Assessment	Test cross-task transfer	Held-out task families, environment variation, toolset diversity	Robustness prediction	Designing controlled similarity
Human-in-the-Loop	Incorporate expert judgment	Hybrid evaluation protocols, interaction interfaces, aggregation methods	Captures subjective quality	Scalability constraints
Architecture-Aware Evaluation	Account for design differences	Architecture-specific metrics, stress tests, fair comparison protocols	Targeted capability assessment	Risk of overfitting to architectures

8.3 Barriers and open challenges for emerging evaluation paradigms

Although the trends are promising, there are still major challenges that hinder the adoption of the next-generation agent evaluation. The cost of in-depth assessment and high cost of calculation and money restrict access to in-depth assessment making it possible that in a situation where the rich are getting richer, a team with the money to buy lots of measurements can make their iteration quicker. Stochastic, interactive environments raise reproducibility challenges that hamper scientific rigour and makes fair comparison of research groups across different, temporally or structurally varying, infrastructural settings difficult.

The inability to standardize safety measures is indicative of more conceptual issues: safety is contextual, it deals with low-probability high-consequence events which are difficult to measure empirically, and is necessitated by predicting new failure modes not observable in existing systems. The conflict between realism and experimental control has not been completely resolved—more real world assessments are lost in the name of reproducibility and systematic variation, whereas controlled experiments are lost in the name of ecological validity.

Lastly, benchmark overfitting and leader board gaming still pose a threat to evaluation integrity. By going public and making benchmarks highly optimized against, they run the risk of measuring system-specific optimization instead of capability that is generalized. Dynamic benchmarks including periodically updated tasks are partial but come with their fair share of issues regarding longitudinal comparability and cost of evaluation.

9 Conclusion

This review analyzed the assessment scenario of agentic AI systems critically, analyzing benchmarks, metrics, datasets, and methodologies in a broad range of applications. The key conclusion is obvious: the modern practices of evaluation can be essentially unable to address dimensions of agent behavior that are critical in deployment. Although frameworks like AgentBench, WebArena, SWE-bench, PaperBench, MLGym, and BrowserGym take the field to the next level of no longer just the assessment of specific tasks in a static manner, they are nonetheless characterized by binary success measures that do not reflect the alignment of planning, tool-use efficiency, memory management, cost-efficiency, and safety compliance. This results-oriented bias gives rise to a systemic evaluation-implementation discrepancy in which benchmark performance is a significant poor predictor of actual viability. These challenges are crystallized in the form of software development agents. Test-passing measures have been shown to be excellent in measuring functional correctness and systematically neglecting the code quality, security vulnerabilities, maintainability and integration of workflows that are vital in the production deployment. This kind of failure is common to web automation, embodied agents, and scientific fields of research and, in all cases, it is the evaluation methodology rather than model capability that represents the major bottleneck. Architecturally different agent systems ReAct, Reflexion, multi-agent frameworks have radically different failure modes that cannot be distinguished by architecture-agnostic benchmarks. The trends of realistic environments, lifelong assessment, analysis at the trajectory level, and safety-conscious metrics have a potential, but there are still critical obstacles: high computational costs, the impossibility of reproducibility, and the

lack of standard taxonomies. Strict evaluation infrastructure is essential in the development of agentic AI. The community can use principled, multi-dimensional, deployment-oriented evaluation as the only way to make sure that improvements in agent capabilities can be converted to reliable, safe, and useful systems in the real world.

In summary, this review demonstrates that evaluation methodology not model capability is the primary barrier to trustworthy agentic AI deployment. Across all 15 benchmarks examined, the evidence is unambiguous: safety is unscored (0/15), cost is untracked (0/15), trajectory quality is unmeasured in 14/15 cases, and binary success metrics are the sole evaluation criterion in 13/15 benchmarks. No benchmark currently exceeds 50% on our deployment-readiness rubric (Table 10). Until evaluation infrastructure evolves to capture these dimensions, benchmark leaderboard rankings will remain poor proxies for real-world viability. We call on the research community to treat evaluation methodology as a first-class scientific contribution, on equal standing with model development itself.

Acknowledgements This work was supported by the institutional research project Digital Twins and Smart Solutions at the Faculty of Organization and Informatics, University of Zagreb, funded by the European Union – NextGenerationEU.

Author contributions Tanzila Kehkashan, Muhammad Abdullah, and Abdul Rehman contributed equally to this work. Tanzila Kehkashan led the study conceptualization and manuscript drafting. Muhammad Abdullah contributed to the literature review and evaluation analysis. Abdul Rehman supported analysis and validation. Ahmad Sami Al-Shamayleh and Nikola Ivković contributed to interpretation and manuscript revision. Nor Azman Ismail and Sharifah Sakinah Syed Ahmad provided supervision and methodological guidance. Adnan Akhunzada supervised the research and performed the final review and approval. All authors read and approved the final manuscript.

Data availability No datasets were generated or analysed during the current study.

Declarations

Conflict of interest The authors declare no Conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Acharya DB, Kuppan K, Divya B (2025) Agentic ai: Autonomous intelligence for complex goals—a comprehensive survey. IEEe Access
- Akshathala S, Adnan B, Ramesh M, Vaidhyanathan K, Muhammed B, Parthasarathy K (2025) Beyond task completion: An assessment framework for evaluating agentic ai systems. arXiv preprint [arXiv:2512.12791](https://arxiv.org/abs/2512.12791)
- Almeida Y, Albuquerque D, Dantas Filho E, Muniz F, Farias Santos K, Perkusich M, Almeida H, Perkusich A (2024) Aicodereview: Advancing code quality with ai-enhanced reviews. *SoftwareX* 26:101677

- Anwar R, Bashir MB (2023) A systematic literature review of ai-based software requirements prioritization techniques. *IEEE Access* 11:143815–143860
- Arabiat A, Altaeyeb M (2024) Enhancing internet of things security: evaluating machine learning classifiers for attack prediction. *Int J Elect Comput Eng* 14(5):52
- Arabiat A, Hassan M, Almomani O (2023) Weka-based machine learning for traffic congestion prediction in amman city. *Int J Artif Intell ISSN* 2252(8938):4423
- Baack S (2024) A critical analysis of the largest source for generative ai training data: Common crawl. In: *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency*, pp. 2199–2208
- Bandi A, Kongari B, Naguru R, Pasnoor S, Vilipala SV (2025) The rise of agentic ai: a review of definitions, frameworks, architectures, applications, evaluation metrics, and challenges. *Fut Intern* 17(9):404
- Bayoudh K, Knani R, Hamdaoui F, Mtibaa A (2022) A survey on deep multimodal learning for computer vision: advances, trends, applications, and datasets. *Vis Comput* 38(8):2939–2970
- Borazio F, Croce D, Basili R (2025) Unitor at bioasq 2025: modular biomedical qa with synthetic snippets and multiple task answer generation. In: *CLEF*
- Brohan A, Chebotar Y, Finn C, Hausman K, Herzog A, Ho D, Ibarz J, Irpan A, Jang E, Julian R, et al (2023) Do as i can, not as i say: Grounding language in robotic affordances. In: *Conference on Robot Learning*, pp. 287–318. PMLR
- Canese L, Cardarilli GC, Di Nunzio L, Fazzolari R, Giardino D, Re M, Spanò S (2021) Multi-agent reinforcement learning: a review of challenges and applications. *Appl Sci* 11(11):4948
- Chang Y, Wang X, Wang J, Wu Y, Yang L, Zhu K, Chen H, Yi X, Wang C, Wang Y et al (2024) A survey on evaluation of large language models. *ACM Trans Intell Syst Technol* 15(3):1–45
- Chen X, Hu X, Huang Y, Jiang H, Ji W, Jiang Y, Jiang Y, Liu B, Liu H, Li X et al (2025) Deep learning-based software engineering: progress, challenges, and opportunities. *Sci China Inf Sci* 68(1):111102
- Chen Y, Xie H, Ma M, Kang Y, Gao X, Shi L, Cao Y, Gao X, Fan H, Wen M, et al (2024) Automatic root cause analysis via large language models for cloud incidents. In: *Proceedings of the Nineteenth European Conference on Computer Systems*, pp. 674–688
- Chen M (2021) Evaluating large language models trained on code. *arXiv preprint* [arXiv:2107.03374](https://arxiv.org/abs/2107.03374)
- Cihon P, Stein M, Bansal G, Manning S, Xu K (2025) Measuring ai agent autonomy: Towards a scalable approach with code inspection. *arXiv preprint* [arXiv:2502.15212](https://arxiv.org/abs/2502.15212)
- Dakhel AM, Nikanjam A, Majdinasab V, Khomh F, Desmarais MC (2024) Effective test generation using pre-trained large language models and mutation testing. *Inf Softw Technol* 171:107468
- Deng X, Gu Y, Zheng B, Chen S, Stevens S, Wang B, Sun H, Su Y (2023) Mind2web: towards a generalist agent for the web. *Adv Neural Inf Process Syst* 36:28091–28114
- Dong Y, Ding J, Jiang X, Li G, Li Z, Jin Z (2025) Codescore: evaluating code generation by learning code execution. *ACM Trans Softw Eng Methodol* 34(3):1–22
- Dong Y, Jiang X, Jin Z, Li G (2024) Self-collaboration code generation via chatgpt. *ACM Trans Softw Eng Methodol* 33(7):1–38
- Du W, Ding S (2021) A survey on multi-agent deep reinforcement learning: from the perspective of challenges and applications. *Artif Intell Rev* 54(5):3215–3238
- Eisty NU, Carver JC (2022) Testing research software: a survey. *Empir Softw Eng* 27(6):138
- Esposito G (2025) Llm in the siem loop: A contract-based framework for threat detection with an evaluation on windows telemetry and mitre att&ck mapping. PhD thesis, Politecnico di Torino
- Evtikhiev M, Bogomolov E, Sokolov Y, Bryksin T (2023) Out of the bleu: how should we assess quality of the code generation models? *J Syst Softw* 203:111741
- Gemp I, Anthony T, Bachrach Y, Bhoopchand A, Bullard K, Connor J, Dasagi V, De Vylder B, Duenez-Guzman EA, Elie R et al (2022) Developing, evaluating and scaling learning agents in multi-agent environments. *AI Commun* 35(4):271–284
- Geva M, Khashabi D, Segal E, Khot T, Roth D, Berant J (2021) Did aristotle use a laptop? A question answering benchmark with implicit reasoning strategies. *Trans Assoc Comput Ling* 9:346–361
- Golchin S, Surdeanu M (2023) Time travel in llms: Tracing data contamination in large language models. *arXiv preprint* [arXiv:2308.08493](https://arxiv.org/abs/2308.08493)
- Gulli A (2025) *AI Agents on the CLI*. Springer, Cham, pp 381–386
- Gundersen OE, Shamsaliei S, Isdahl RJ (2022) Do machine learning platforms provide out-of-the-box reproducibility? *Futur Gener Comput Syst* 126:34–47
- Guo H, Chen X, Huang Y, Wang Y, Ding X, Zheng Z, Zhou X, Dai H-N (2023) Snippet comment generation based on code context expansion. *ACM Trans Softw Eng Methodol* 33(1):1–30
- He J, Treude C, Lo D (2025) Llm-based multi-agent systems for software engineering: literature review, vision, and the road ahead. *ACM Trans Softw Eng Methodol* 34(5):1–30
- Hendrycks D, Carlini N, Schulman J, Steinhardt J (2021) Unsolved problems in ml safety. *arXiv preprint* [arXiv:2109.13916](https://arxiv.org/abs/2109.13916)

- Hou X, Zhao Y, Liu Y, Yang Z, Wang K, Li L, Luo X, Lo D, Grundy J, Wang H (2024) Large language models for software engineering: A systematic literature review. *ACM Trans Softw Eng Methodol* 33(8):1–79
- Hu K, Li M, Song Z, Xu K, Xia Q, Sun N, Zhou P, Xia M (2024) A review of research on reinforcement learning algorithms for multi-agents. *Neurocomputing* 599:128068
- Huang Y, Huang J, Chen X, He K, Zhou X (2023) Bcgen: a comment generation method for bytecode. *Autom Softw Eng* 30(1):5
- Hui DY-T, Chevalier-Boisvert M, Bahdanau D, Bengio Y (2020) Babyai 1.1. arXiv preprint [arXiv:2007.12770](https://arxiv.org/abs/2007.12770)
- Jimenez CE, Yang J, Wettig A, Yao S, Pei K, Press O, Narasimhan K (2023) Swe-bench: Can language models resolve real-world github issues? arXiv preprint [arXiv:2310.06770](https://arxiv.org/abs/2310.06770)
- Kamalloo E, Gontier N, Lu XH, Dziri N, Murty S, Lacoste A (2025) Proceedings of the 1st workshop for research on agent language models (realm 2025). In: *Proceedings of the 1st Workshop for Research on Agent Language Models (REALM 2025)*
- Kapoor S, Narayanan A (2023) Leakage and the reproducibility crisis in machine-learning-based science. *Patterns* 4(9):123
- Kazdan J, Schaeffer R, Allouah Y, Sullivan C, Yu K, Levi N, Koyejo S (2025) Efficient prediction of pass@k scaling in large language models. arXiv preprint [arXiv:2510.05197](https://arxiv.org/abs/2510.05197)
- Kenton Z, Everitt T, Weidinger L, Gabriel I, Mikulik V, Irving G (2021) Alignment of language agents. arXiv preprint [arXiv:2103.14659](https://arxiv.org/abs/2103.14659)
- Kim J, Shin B, Chung J, Rhu M (2025) The cost of dynamic reasoning: Demystifying ai agents and test-time scaling from an ai infrastructure perspective. arXiv preprint [arXiv:2506.04301](https://arxiv.org/abs/2506.04301)
- Kio G (2025) Swe-bench-secret: Automating ai agent evaluation for software engineering tasks
- Lebovitz S, Levina N, Lifshitz-Assaf H (2021) Is ai ground truth really true? the dangers of training and evaluating ai tools based on experts' know-what. *MIS Q* 45(3):1501–1526
- Li S, Puig X, Paxton C, Du Y, Wang C, Fan L, Chen T, Huang D-A, Akyürek E, Anandkumar A et al (2022) Pre-trained language models for interactive decision-making. *Adv Neural Inf Process Syst* 35:31199–31212
- Li M, Zhao Y, Yu B, Song F, Li H, Yu H, Li Z, Huang F, Li Y (2023) Api-bank: A comprehensive benchmark for tool-augmented llms. arXiv preprint [arXiv:2304.08244](https://arxiv.org/abs/2304.08244)
- Li D, Murr L (2024) Humaneval on latest gpt models–2024. arXiv preprint [arXiv:2402.14852](https://arxiv.org/abs/2402.14852)
- Li Z, Li Y, Zhang Y, et al (2024) Adaptive environmental modeling for task-oriented language agents
- Li Z, Zhao X, Cui J, Shen Z (2025) A frustratingly simple yet highly effective attack baseline: Over 90% success rate against the strong black-box models of gpt-4.5/4o/o1. arXiv preprint [arXiv:2503.10635](https://arxiv.org/abs/2503.10635)
- Liang P, Bommasani R, Lee T, Tsipras D, Soylu D, Yasunaga M, Zhang Y, Narayanan D, Wu Y, Kumar A, et al (2022) Holistic evaluation of language models. arXiv preprint [arXiv:2211.09110](https://arxiv.org/abs/2211.09110)
- Liu X, Yu H, Zhang H, Xu Y, Lei X, Lai H, Gu Y, Ding H, Men K, Yang K, et al (2023) Agentbench: Evaluating llms as agents. arXiv preprint [arXiv:2308.03688](https://arxiv.org/abs/2308.03688)
- Lyu Z, Li X, Xie Z, Li M (2025) Top pass: improve code generation by pass@k-maximized code ranking. *Front Comp Sci* 19(8):198341
- Martínez-Fernández S, Bogner J, Franch X, Oriol M, Siebert J, Trendowicz A, Vollmer AM, Wagner S (2022) Software engineering for ai-based systems: a survey. *ACM Trans Softw Eng Methodol (TOSEM)* 31(2):1–59
- Merrill MA, Shaw AG, Carlini N, Li B, Raj H, Bercovich I, Shi L, Shin JY, Walshe T, Buchanan EK, et al (2026) Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. arXiv preprint [arXiv:2601.11868](https://arxiv.org/abs/2601.11868)
- Mialon G, Fourrier C, Wolf T, LeCun Y, Scialom T (2023) Gaia: a benchmark for general ai assistants. In: *The Twelfth International Conference on Learning Representations*
- Mohammadi M, Li Y, Lo J, Yip W (2025) Evaluation and benchmarking of llm agents: A survey. In: *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, pp. 6129–6139
- Mohan A, Zhang A, Lindauer M (2024) Structure in deep reinforcement learning: a survey and open problems. *J Artif Intell Res* 79:1167–1236
- Moreno AC, Bitterman DS (2024) Toward clinical-grade evaluation of large language models. *Int J Radiat Oncol Biol Phys* 118(4):916–920
- Munikoti S, Agarwal D, Das L, Halappanavar M, Natarajan B (2023) Challenges and opportunities in deep reinforcement learning with graph neural networks: a comprehensive review of algorithms and applications. *IEEE Trans Neural Netw Learn Syst* 35(11):15051–15071
- Nair SS, Muddapu VR, Vigneshwaran C, Balasubramani PP, Ramanathan DS, Mishra J, Chakravarthy VS (2023) A generalized reinforcement learning based deep neural network agent model for diverse cognitive constructs. *Sci Rep* 13(1):5928
- Nashaat M, Miller J (2024) Towards efficient fine-tuning of language models with organizational data for automated software review. *IEEE Transactions on Software Engineering*

- Nathani D, Madaan L, Roberts N, Bashlykov N, Menon A, Moens V, Budhiraja A, Magka D, Vorotilov V, Chaurasia G, et al (2025) Mlgym: A new framework and benchmark for advancing ai research agents. arXiv preprint [arXiv:2502.14499](https://arxiv.org/abs/2502.14499)
- Ndukwe IG, Licorish SA, Tahir A, MacDonell SG (2023) How have views on software quality differed over time? Research and practice viewpoints. *J Syst Softw* 195:111524
- Nguyen TT, Nguyen ND, Nahavandi S (2020) Deep reinforcement learning for multiagent systems: a review of challenges, solutions, and applications. *IEEE Trans Cybern* 50(9):3826–3839
- Ott S, Barbosa-Silva A, Blagec K, Brauner J, Samwald M (2022) Mapping global dynamics of benchmark creation and saturation in artificial intelligence. *Nat Commun* 13(1):6793
- Pineau J, Vincent-Lamarre P, Sinha K, Larivière V, Beygelzimer A, d'Alché-Buc F, Fox E, Larochelle H (2021) Improving reproducibility in machine learning research (a report from the neurips 2019 reproducibility program). *J Mach Learn Res* 22(164):1–20
- Raff E (2019) A step toward quantifying independently reproducible machine learning research. *Adv Neural Inf Process Syst* 32:120
- Rahman H, D'Cruze RS, Ahmed MU, Sohlberg R, Sakao T, Funk P (2022) Artificial intelligence-based life cycle engineering in industrial production: a systematic literature review. *IEEE Access* 10:133001–133015
- Sankhe MP, Patil N, Ghorpade MM, Prasad MP, Linkesh MM (2025) Empirical analysis of ai-assisted code generation tools: Impact on code quality, security and developer productivity
- Satav A (2025) Enterprise api & platform strategy in the era of agentic ai. *J Comput Sci Technol Stud* 7(1):380–385
- Schäfer M, Nadi S, Eghbali A, Tip F (2023) An empirical evaluation of using large language models for automated unit test generation. *IEEE Trans Software Eng* 50(1):85–105
- Shafiq S, Mashkoor A, Mayr-Dorn C, Egyed A (2021) A literature review of using machine learning in software development life cycle stages. *IEEe Access* 9:140896–140920
- Shridhar M, Thomason J, Gordon D, Bisk Y, Han W, Mottaghi R, Zettlemoyer L, Fox D (2020) Alfred: A benchmark for interpreting grounded instructions for everyday tasks. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 10740–10749
- Singh AK, Hsieh S-H (2026) Multi-llm-based augmentation and synthetic data generation of construction schedules and task descriptions with slm-as-a-judge assessment. *Adv Eng Inform* 69:103825
- Sivamayil K, Rajasekar E, Aljafari B, Nikolovski S, Vairavasundaram S, Vairavasundaram I (2023) A systematic study on reinforcement learning based applications. *Energies* 16(3):1512
- Starace G, Jaffe O, Sherburn D, Aung J, Chan JS, Maksin L, Dias R, Mays E, Kinsella B, Thompson W, et al (2025) Paperbench: Evaluating ai's ability to replicate ai research. arXiv preprint [arXiv:2504.01848](https://arxiv.org/abs/2504.01848)
- Sumers T, Yao S, Narasimhan K, Griffiths T (2023) Cognitive architectures for language agents. *Transactions on Machine Learning Research*
- Tarasova N, Balp-Straffon E, Iancheruk A, Sielskiy Y, Kozodoi N, Byrne LH, Butler J, Czelej M, Ang A, Shah Y, et al (2025) Swe-infrabench: Evaluating language models on cloud infrastructure code. In: *NeurIPS 2025 Workshop on Evaluating the Evolving LLM Lifecycle: Benchmarks, Emergent Abilities, and Scaling*
- Terragni V, Vella A, Roop P, Blincoc K (2025) The future of ai-driven software engineering. *ACM Trans Softw Eng Methodol* 34(5):1–20
- Thakkar M, Chapados N, Pal C, et al (2025) Webarena verified: Reliable evaluation for web agents. In: *Workshop on Scaling Environments for Agents*
- Wachter C, Beckschulte S, Hinrichs MP, Sohnius F, Schmitt RH (2024) Strategies for resilient manufacturing: a systematic literature review of failure management in production. *Procedia CIRP* 130:1393–1402
- Wang J, Huang Y, Chen C, Liu Z, Wang S, Wang Q (2024) Software testing with large language models: Survey, landscape, and vision. *IEEE Trans Software Eng* 50(4):911–936
- Wang L, Ma C, Feng X, Zhang Z, Yang H, Zhang J, Chen Z, Tang J, Chen X, Lin Y et al (2024) A survey on large language model based autonomous agents. *Front Comp Sci* 18(6):186345
- Wang Y, Ma X, Zhang G, Ni Y, Chandra A, Guo S, Ren W, Arulraj A, He X, Jiang Z et al (2024) Mmlu-pro: a more robust and challenging multi-task language understanding benchmark. *Adv Neural Inf Process Syst* 37:95266–95290
- Wang G, Liu J, Zhou M, Chen X, Zhang L, Sun Z (2025) Toolbench 2.0: Evaluating long-horizon and multi-step tool use in llms
- WebChoreArena (2025) WebArena: Webchorearena: Evaluating web browsing agents on realistic tedious web tasks. arXiv preprint
- Xu FF, Vasilescu B, Neubig G (2022) In-ide code generation from natural language: Promise and challenges. *ACM Trans Softw Eng Methodol (TOSEM)* 31(2):1–47

- Xu F, Medappa PK, Tunc MM (2024) Does ai technology deployment benefit the owner of the technology? impact of github copilot release on microsoft. *Impact of GitHub Copilot Release on Microsoft* (June 15, 2024)
- Yan Y, Li J, Zaggia C (2025) The analysis of deep reinforcement learning for dynamic graphical games under artificial intelligence. *Sci Rep* 15(1):23133
- Yang G-Y, Wang F (2025) Taming silent failures: a framework for verifiable ai reliability. *IEEE Reliabil Magaz* 2(4):46–55
- Yang Z, Qi P, Zhang S, Bengio Y, Cohen W, Salakhutdinov R, Manning CD (2018) Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In: *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pp. 2369–2380
- Yeo S, Ma Y-S, Kim SC, Jun H, Kim T (2024) Framework for evaluating code generation ability of large language models. *ETRI J* 46(1):106–117
- Yu C, Liu J, Nemati S, Yin G (2021) Reinforcement learning in healthcare: a survey. *ACM Comput Surv (CSUR)* 55(1):1–36
- Yu Z, Zhao Y, Cohan A, Zhang X-P (2024) Humaneval pro and mbpp pro: Evaluating large language models on self-invoking code generation. *arXiv preprint arXiv:2412.21199*
- Zhang T, Mo H (2021) Reinforcement learning for robot research: A comprehensive review and open issues. *Int J Adv Rob Syst* 18(3):17298814211007304
- Zhou S, Xu FF, Zhu H, Zhou X, Lo R, Sridhar A, Cheng X, Ou T, Bisk Y, Fried D, et al (2023) Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Tanzila Kehkashan^{1,2} · Muhammad Abdullah² · Ahmad Sami Al-Shamayleh³ · Nikola Ivković⁴ · Nor Azman Ismail¹ · Sharifah Sakinah Syed Ahmad⁵ · Abdul Rehman² · Adnan Akhunzada⁶ 

✉ Muhammad Abdullah
malikabdmalik468@gmail.com

✉ Adnan Akhunzada
adnan.adnan@udst.edu.qa

Tanzila Kehkashan
tanzila.kehkashan@gmail.com

Ahmad Sami Al-Shamayleh
a.alshamayleh@ammanu.edu.jo

Nikola Ivković
nikola.ivkovic@foi.hr

Nor Azman Ismail
azman@utm.my

Sharifah Sakinah Syed Ahmad
sakinah@utem.edu.my

Abdul Rehman
m.abdulrehman.shoukat@gmail.com

¹ Faculty of Computing, Universiti Teknologi Malaysia, Johor Bahru 81310, Malaysia

² Faculty of Information Technology, University of Lahore, Sargodha 40100, Pakistan

³ Department of Data Science and Artificial Intelligence, Faculty of Information Technology, Al-Ahliyya Amman University, Amman 19328, Jordan

- ⁴ Faculty of Organization and Informatics, University of Zagreb, Varaždin 42000, Croatia
- ⁵ Faculty of Artificial Intelligence and Cyber Security, Universiti Teknikal Malaysia Melaka, 76100 Durian Tunggal, Melaka,, Malaysia
- ⁶ College of Computing & IT, Department of Data and Cybersecurity, University of Doha for Science and Technology, Doha 24449, Qatar